

UNIVERSITY OF BUEA

P.O Boc 63,
Buea, South West Region
CAMEROON
Tel: (237) 3332 21 34/ 3332 26 90
Fax: (237) 3332 22 72



REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

DESIGN AND IMPLEMENTATION OF A SCHOOL ATTENDANCE MANAGEMENT SYSTEM

*A dissertation submitted to the Department of Computer Engineering,
Faculty of Engineering and Technology, University of Buea, in Partial
Fulfilment of the Requirements for the Award of Bachelor of Engineering
(B.Eng.) Degree in Computer Engineering.*

By:

TEGUE NONO MIKEL MODEIRO

Matriculation Number: FE21A321

Option: Software Engineering

Supervisor

Dr. ALINE TSAGUE

University of Buea

2024/2025 Academic Year

CERTIFICATE OF ORIGINALITY

We the undersigned, hereby certify that this dissertation, entitled: “DESIGN AND IMPLEMENTATION OF A SCHOOL ATTENDANCE MANAGEMENT SYSTEM” presented by: TEGUE NONO MIKEL MODEIRO, FE21A321. Have been carried out by him in the Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea under the supervision of Dr. ALINE TSAGUE. This dissertation is authentic and represents the fruits of his own research and efforts.

Date: _____

Student: _____ Supervisor _____

Head of Department _____

DEDICATION

It's with the deepest gratitude and admiration that this work is lovingly dedicated to the TEGUE family.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my academic supervisor, Dr. ALINE TSAGUE, who has been more than just a mentor, a true maternal figure whose guidance, encouragement, and patience meant so much to me throughout this journey. I am also thankful to the University of Buea, particularly the Faculty of Engineering and Technology, and the Head of Department, Prof. ELIE FUTE, for providing a solid academic foundation.

Special thanks to my lecturers; Dr. SOP DEFFO, Dr. NKEMENI VALERY, Dr. NGUTI, Dr. FOZIN, AND Dr. INES, for the knowledge they've passed on, and for their continued support and wise counsel during my four years of study.

I deeply appreciate my beloved family. To my mother, Mme. TEGUE FOSSO MAKEU MELANIE, and father, Mr. TAGNE TEGUE VICTOR, thank you for your love, sacrifice, trust, and unwavering belief in me. Your strength and hard work gave me the foundation to pursue this dream. To my brothers; TEGUE ALPHAND BROWN, NAMI FRANK DILANE, TEGUE LAWRENCE CHANA, TEGUE EMILIE, AND MATENE MERVEILLE, thank you for your constant support, encouragement, and love.

To my wonderful friends who stood by me like a second family, I am truly grateful: DJITUE BRINDA, ATSAFAC AGAFINA, FOFIE ELISABETH, NTAMO PATRICIA, NEGUE GRACE, NGUEPI PATERSON, LONCHI JORDAN, STEPHANE MERCI, NEBA PRINCEWILL, AMARACHUKWU GODLOVE, KAMDEM GILLES CHRISTIAN, KAH JOSPEN, NGULEFAC JERRY, BILAL, OLIVIER, NYAMBE BIONIC, NYAMBE BORIS, MAXWELL TRICAGO, FRANK DILANE NGONGANG, WAFFO FRANKLIN, YANLAP SANDRA, LEMOUPA FRANK (your help on this project meant everything), RAMSEY, TAMBE BORIS, CHEZEM MIGUEL, KOUAM WATI ZIDANE, LANDRY, REGAN, WOUWE FABIOLA, TOUATSAP ULYSSE, JOHANNA EUNICE, DANN CYNDY AND HAMMIEL. I'm grateful for the moments we shared and the strength I drew from each of you.

Finally, I give all thanks and glory to the Almighty God, who made this journey possible. His grace, protection, and constant presence have carried me through every challenge and every success. Without Him, none of this would have been possible.

ABSTRACT

In many Cameroonian educational institutions, the management of student attendance is still largely done manually through paper registers or spreadsheets, a practice that is increasingly becoming ineffective. These traditional methods are time-consuming, prone to human errors, and susceptible to manipulation and data loss. As academic institutions strive for digital transformation and improved administrative efficiency, there is an urgent need for a reliable, secure, and automated attendance tracking system that aligns with modern educational practices. This project addresses the challenge by proposing and implementing a mobile-based attendance management system that leverages biometric verification, geofencing, and blockchain technology to ensure accurate, tamper-proof attendance tracking. The system is designed with the realities and constraints of Cameroonian institutions in mind. Through a comprehensive requirements engineering process involving key stakeholders; students, lecturers, and administrative staff, the specific needs and expectations were identified. These include the ability to monitor attendance in real time, provide secure authentication, maintain student privacy, prevent impersonation, and generate reliable attendance statistics for both students and decision-makers.

Keywords:

TABLE OF CONTENT

CERTIFICATE OF ORIGINALITY	II
DEDICATION	III
ACKNOWLEDGEMENT	IV
ABSTRACT.....	V
TABLE OF CONTENT	VI
LIST OF TABLES	IX
LIST OF FIGURES	X
LIST OF ABBREVIATIONS.....	XII
CHAPTER 1: GENERAL INTRODUCTION	1
1.1. BACKGROUND AND CONTEXT OF THE STUDY	1
1.2. PROBLEM STATEMENT	1
1.3. OBJECTIVES OF THE STUDY	4
1.3.1. General Objective	4
1.3.2. Specific Objectives	4
1.4. PROPOSED METHODOLOGY	5
1.5. RESEARCH QUESTIONS	5
1.6. RESEARCH HYPOTHESIS	6
1.7. SIGNIFICANCE OF THE STUDY	6
1.8. SCOPE OF THE STUDY	6
1.9. DELIMITATION OF THE STUDY	7
1.10. DEFINITION OF KEYWORDS AND TERMS	7
1.10. ORGANIZATION OF THE DISSERTATION.....	8
CHAPTER 2: LITERATURE REVIEW	10
2.1. INTRODUCTION	10
2.2. GENERAL CONCEPTS	10
2.2.1. Blockchain Technology	10
2.2.2. Geofencing Technology.....	11
2.2.3. Attendance Management System.....	11

2.2.4. Blockchain-Based Attendance Management System	11
2.2.5. Peer-To-Peer (P2p) System.....	11
2.2.6. Permissioned Ledger.....	11
2.2.7. Consensus Mechanism.....	11
2.2.8. Smart Contracts.....	11
2.2.9. Types Of Nodes	11
2.2.10. Channels In Hyperledger Fabric	12
2.2.11. Organizations In Hyperledger Fabric.....	12
2.3. RELATED WORKS.....	12
2.4. PARTIAL CONCLUSION.....	16
CHAPTER 3: ANALYSIS AND DESIGN	17
3.1. INTRODUCTION	17
3.2. METHODOLOGY	17
3.2.1. AGILE SDLC MODEL	17
3.2.2. REQUIREMENT ENGINEERING.....	19
3.2.3. SRS DOCUMENT.....	27
3.3. SYSTEM DESIGN	30
3.3.1. HIGH-LEVEL DESIGN.....	30
3.3.2. LOW-LEVEL DESIGN.....	43
3.4 . GLOBAL ARCHITECTURE OF THE SOLUTION	49
3.5. DESCRIPTION OF THE ALGORITHMS	49
3.6. DESCRIPTION OF THE RESOLUTION PROCESS	52
3.7. PARTIAL CONCLUSION.....	52
CHAPTER 4: IMPLEMENTATION & RESULTS	53
4.1. INTRODUCTION	53
4.2. TOOLS AND MATERIALS USED.....	53
4.3. DESCRIPTION OF IMPLEMENTATION PROCESS	54
4.3.1. Frontend Implementation Mobile App (Expo/React Native)	54
4.3.2. Backend Implementation Node.js / Express	56
4.3.3. Blockchain Network Integration.....	61
4.4. PRESENTATION AND INTERPRETATION OF RESULTS.....	66
4.5. EVALUATION OF SOLUTION	70
PARTIAL CONCLUSION.....	72

CHAPTER 5: CONCLUSION AND FURTHER WORKS	73
5.1. SUMMARY OF FINDINGS	73
5.2. CONTRIBUTION TO ENGINEERING AND TECHNOLOGY	73
5.3. RECOMMENDATION	73
5.4. DIFFICULTIES ENCOUNTERED	75
5.5. FURTHER WORKS	76
REFERENCES	77
APPENDIX.....	79

LIST OF TABLES

Table 1: requirement gathering roles assignment	21
Table 2: requirement evaluation & synthesis.....	26

LIST OF FIGURES

Figure 1: Haversine formula	14
Figure 2: Agile Model.....	17
Figure 3: requirement engineering process.....	19
Figure 4: requirement gathering process.....	20
Figure 5: end-to-end interview 1.....	22
Figure 6: end-to-end Interview 2	22
Figure 7: end-to-end Interview 3	23
Figure 8: requirement survey	24
Figure 9: students survey stats	24
Figure 10: survey open answers.....	25
Figure 11: requirement analysis.....	25
Figure 12: sequence of user authentication.....	32
Figure 13: sequence of on-chain enrollment and clock-in.....	34
Figure 14: student use cases.....	36
Figure 15: instructor use cases.....	39
Figure 16: system auto use cases	41
Figure 17: class diagram	44
Figure 18: state machine diagram	45
Figure 19: activity diagram.....	46
Figure 20: ER-Diagram.....	47
Figure 21: Relational model (schema).....	48
Figure 22: Global System Architecture	49
Figure 23: Ray casting Pseudo code	51
Figure 24: Chain-code pseudo code.....	51
Figure 25: fabric client module.....	57
Figure 26: system-data module	58
Figure 27: bash command to download fabric executables	62
Figure 28: docker-compose.yaml script.....	63
Figure 29: crypto configuration	63
Figure 30: bash command to generate crypto materials	63
Figure 31: configtx.yaml file	64

Figure 32: generate genesis block.....	64
Figure 33: contract-api application (chain-code).....	64
Figure 34: bash command for network init.....	65
Figure 35: bash command for chaincode deployment.....	65
Figure 36: network init result.....	66
Figure 37: chaincode packaging	67
Figure 38: chaincode definition approvals.....	67
Figure 39: chain code well deployed	67
Figure 40: network containers up and running	68
Figure 41: splash screen and login.....	68
Figure 42: attendance recorded.....	69
Figure 43: course-session selection and clockin.....	69
Figure 44: admin-panel.....	70

LIST OF ABBREVIATIONS

CHAPTER 1: GENERAL INTRODUCTION

1.1. BACKGROUND AND CONTEXT OF THE STUDY

In most educational institutions across Cameroon, attendance management is still handled manually. Lecturers or class delegates typically use paper-based registers or printed Excel sheets to record student presence. This method is not only time-consuming but also prone to errors, manipulation, and loss of data. For example, attendance sheets can be tampered with, destroyed, or misplaced, which raises concerns about the integrity of academic records.

With the growing number of students in higher institutions, it has become increasingly difficult to track and manage attendance efficiently. Additionally, the current systems provide little to no real-time monitoring or verification, making it easy for students to falsify their attendance, especially when they are not physically present in class. This lack of control and accountability negatively affects both administrative decision-making and students' academic performance records.

Furthermore, there is a growing need for secure and automated systems that can offer features like real-time tracking, accurate location-based verification, personalized attendance statistics, and seamless reporting. The use of fingerprint biometrics and geofencing, combined with modern technologies such as blockchain, offers a promising solution to these problems.

This project aims to explore and develop a mobile-based attendance tracking system that integrates biometric verification, geofencing for location validation, and blockchain technology for data integrity and transparency. The solution will be structured in a way that meets the needs of students, lecturers, and administrative units, while addressing the challenges of the current attendance practices in Cameroon's schools and universities.

1.2. PROBLEM STATEMENT

Attendance management is a critical part of academic administration, yet many educational institutions in Cameroon still rely on outdated, manual methods. These traditional systems are prone to errors, manipulation, and inefficiencies, which affect student performance tracking and institutional reporting. This project seeks to identify and address the root causes of these issues through a more secure and efficient digital solution.

Theme Exploration

- I care about attendance management because it's vital for tracking student participation and academic performance.
- Manual methods used in Cameroon (paper sheets, Excel printouts) are inefficient, prone to loss, and easy to manipulate.
- Class delegates confirm frequent issues: lost sheets, fake signatures, and time wasted during roll call.
- Lecturers complain about tampering, lack of centralized tracking, and delays in compiling attendance reports.
- Students recognize issues like impersonation, lack of real-time monitoring, and no personal statistics.

Problem Statement

Traditional methods such as manual timesheets or punch cards are prone to inaccuracies, time-consuming to process, and susceptible to fraud. They are also cumbersome to manage and do not provide real-time insights into students' presence, making monitoring difficult

The Five Whys

What is the 5 Whys Analysis?

The 5 Whys Analysis is a Lean problem-solving tool designed to identify the root cause of an issue by asking one simple question repeatedly: “Why?” (Serrat, 2025)

Where It Started

This method was introduced by Sakichi Toyoda, the founder of Toyota, and it became a cornerstone of the company’s world-renowned production system. The idea is deceptively simple: by asking “why” five times, you can move past superficial symptoms and drill down to the underlying cause. (community w. , 2025) (Ohno, 1998)

Why It Works (Bulsuk, 2025)

It’s simple: No fancy software or expensive consultants required.

It’s effective: It ensures you don’t waste time fixing symptoms.

It’s adaptable: You can use it for anything, from factory floor issues to office challenges.

ATTENDANCE SYSTEM FIVE WHYS APPROACH

Problem Statement:

Traditional methods such as manual timesheets or punch cards are prone to inaccuracies, time-consuming to process, and susceptible to fraud. They are also cumbersome to manage and do not provide real-time insights into students' presence, making monitoring difficult.

1st Why: Why are traditional attendance methods inaccurate, time-consuming, and susceptible to fraud?

- Because they rely on manual data entry and physical documentation, which can be tampered with, lost, or manipulated.

2nd Why: Why do manual data entry and physical documentation lead to these problems?

- Because there is no automated validation process to verify a student's actual presence.

3rd Why: Why is there no automated validation process?

- Because traditional attendance systems do not leverage digital technologies like biometric authentication, geofencing, or blockchain, which can ensure real-time verification and immutability of attendance records.

4th Why: Why haven't these digital technologies been implemented in traditional systems?

- Because legacy systems and school infrastructures were built without considering automation and security.

5th Why: Why do institutions hesitate to adopt automated attendance systems?

- Because there is a lack of awareness about the long-term benefits, perceived high implementation costs, and concerns about data privacy.

Root Cause Identified

Resistance to Technological Adoption in Attendance Systems.

The core issue is institutional resistance to adopting digital attendance solutions due to lack of awareness, cost concerns, and integration challenges. Schools and organizations fail to see the long-term benefits of automation, leading to continued reliance on inefficient, error-prone manual processes.

Solution Approach:

To overcome this, institutions must:

- Educate decision-makers on the long-term benefits (reduced fraud, improved accuracy, real-time insights).
- Implement cost-effective solutions like blockchain-based attendance with geofencing to ensure secure and automated tracking.
- Ensure seamless integration with existing school management systems.
- Address privacy concerns through robust security measures and compliance with data protection regulations.

Solution Hypothesis (Problem Summary)

MarkX will solve the problem of inaccurate, time-consuming, and fraud-prone traditional attendance methods by using geofencing technology for secure, real-time, and decentralized attendance verification.

1.3. OBJECTIVES OF THE STUDY

1.3.1. General Objective

Develop an autonomous attendance management system that uses geofencing and decentralized verification to reduce fraud, improve accuracy, and cut attendance recording time by at least 50% in comparison to the traditional paper-based system within X-months

1.3.2. Specific Objectives

- **Design a User-Centered Interface:** Create an intuitive UI for students, lecturers, based on requirement analysis using feedback from real stakeholders.
- **Implement Geofencing-Based Verification:** Integrate location-based services to ensure students can only mark attendance within a specified classroom boundary.
- **Integrate Decentralized Data Storage:** Use blockchain or decentralized technologies to store attendance logs for immutability and transparency.
- **Enable Real-Time Attendance Monitoring:** Develop real-time data synchronization and dashboards for lecturers and admin units to monitor attendance instantly
- **Automated Attendance Report & Biometric Verification**

1.4. PROPOSED METHODOLOGY

The proposed methodology for the MarkX adopts an Agile development approach guided by the Scrum framework. It involves iterative sprints to design and develop key modules, including geofencing, biometric verification, and blockchain integration, with stakeholder feedback driving refinements. The process starts with requirement gathering via interviews and questionnaires, followed by sprint planning to define tasks. Development leverages tools like Expo/React Native, Node.js/Express, and Hyperledger Fabric on a WSL environment, with testing (UAT and feedback collection). This iterative, collaborative method ensures adaptability, efficiency, and alignment with the goal of reducing attendance recording time by at least 50% while enhancing security and accuracy.

1.5. RESEARCH QUESTIONS

For Students & Lecturers

Expectations and System Requirements

- How do students and lecturers perceive the accuracy and reliability of current attendance tracking methods?
- How does the lack of real-time attendance monitoring impact lecturers' ability to enforce class participation?
- What are and lecturers' opinions on using geofencing and blockchain-based attendance systems?
- What privacy or security concerns do students and lecturers have regarding digital attendance tracking?

Adoption and Engagement

- How should the system handle cases where students forget their devices or have technical issues?
- What training or onboarding process would you prefer to understand how the system works?

For Administration & Management

Functional Expectations and System Design

- What data points should be included in the attendance records (e.g., time, location, method of authentication)?
- What reporting and analytics features should the system provide for decision-making?

Security, Privacy, and Compliance

- What security concerns do you foresee in using a decentralized attendance tracking system?
- What regulatory or institutional compliance requirements should the system meet?
- What policies should be in place to handle attendance disputes or technical failures?
- What are the current security and privacy policies in place for student attendance records?
- What are the key concerns or barriers to adopting a decentralized attendance management system from an administrative perspective?
- What are the institution's compliance requirements regarding attendance tracking, and how well do existing systems meet them? if existing system name it

System Usability and Adoption

- How should the system handle exceptional cases such as student absences due to illness or technical difficulties?

1.6. RESEARCH HYPOTHESIS

Availability of Mobile Phones with GPS Location Services and biometric sensors, Increases Geofencing and biometric verification Adoption

1.7. SIGNIFICANCE OF THE STUDY

This project addresses major issues faced by schools in Cameroon, such as fraudulent attendance practices, time-consuming manual processes, and loss of records. By introducing an autonomous geofencing-based and decentralized attendance management system, the study helps improve data accuracy, security, and operational efficiency. It benefits students, lecturers, and administrative units by saving time, reducing errors, and providing reliable attendance reports for decision-making.

1.8. SCOPE OF THE STUDY

This study focuses on the development, testing, and evaluation of a mobile-based attendance system tailored for higher education institutions in Cameroon. It covers features such as student

authentication, location verification through geofencing, attendance recording, decentralized verification for data integrity, and generation of attendance reports. The scope is limited to the recording and management of class attendance only.

1.9. DELIMITATION OF THE STUDY

The system will be limited to student attendance tracking within a defined campus area using mobile devices. It will not handle other academic management tasks like grading. The study will focus on technical implementation (geofencing, decentralized records) and not on the broader administrative or legal reforms related to attendance policies.

It relies on mobile smartphones with GPS capabilities, so students without compatible devices are excluded.

Internet access is required for full functionality, even though offline capabilities are only partially supported.

Administrative policies and legal compliance are considered based on current practices at selected institutions but may not cover all national standards.

Only classroom attendance is tracked, not overall campus presence or activities.

1.10. DEFINITION OF KEYWORDS AND TERMS

- i. Attendance Management System: A digital platform used to record and manage the attendance of students.
- ii. Geofencing: A technology that uses GPS or RFID to create a virtual boundary around a specific location.
- iii. Decentralized Verification: A method of verifying data without relying on a single central server, often using blockchain.
- iv. Real-time Monitoring: The ability to track attendance as it happens without delays.
- v. Immutable Records: Data that cannot be changed once recorded, ensuring reliability and trustworthiness.

1.10. ORGANIZATION OF THE DISSERTATION

This dissertation is structured into five core chapters, each focusing on specific aspects of the research and development process for the blockchain-based attendance system.

ABSTRACT

The abstract provides a succinct summary of the dissertation, outlining the research problem, objectives, methodology, key findings, and conclusions. It serves as a snapshot of the entire study, enabling readers to quickly grasp the essence of the research

CHAPTER 1: GENERAL INTRODUCTION

This chapter introduces the background of the study, defines the problem, states the research objectives (general and specific), and outlines the methodology used. It also presents research questions and hypotheses, the significance, scope, and delimitations of the study, along with definitions of key terms.

CHAPTER 2: LITERATURE REVIEW

A comprehensive review of related literature, exploring traditional attendance systems, the evolution of attendance technologies, and the foundations of blockchain, geofencing, and biometric verification. It also includes analysis of related systems and highlights their limitations, establishing the research gap this project aims to fill.

CHAPTER 3: ANALYSIS AND DESIGN

This chapter focuses on the system's requirement engineering, high-level and low-level design, and architectural planning. It includes diagrams like use case, activity, ER, and class diagrams to visually represent the structure and behavior of the system. The design integrates geolocation, biometric, and blockchain modules.

CHAPTER 4: IMPLEMENTATION AND RESULTS

Details the tools, technologies, and development process used, including frontend (React Native), backend (Node.js/Express), and blockchain integration (Hyperledger Fabric). It presents screenshots and explanations of the implemented system, evaluates its performance, and compares it to existing solutions.

CHAPTER 5: CONCLUSION AND FURTHER WORKS

Summarizes key findings, the system's contributions to engineering and technology, and provides recommendations for improvement. It also discusses challenges encountered and proposes future enhancements such as real-time monitoring using event-based programming and extended instructor reporting features.

CHAPTER 2: LITERATURE REVIEW

2.1. INTRODUCTION

Effective attendance management is crucial for organizations of all sizes, from small businesses to large corporations and educational institutions. Accurate attendance records are essential for payroll processing, performance evaluation, resource allocation, and ensuring accountability. (Hartono, 2024). Traditional attendance tracking methods, such as manual timesheets or punch cards, are often prone to inaccuracies, time-consuming to process, and susceptible to fraud. These methods can also be cumbersome to manage, especially in organizations with dispersed workforces or flexible work arrangements (Pathak, 2023.). Furthermore, they cannot often provide real-time insights into students' presence, making it difficult for lecturers and administration to monitor attendance patterns and identify potential issues. The advent of web-based technologies has opened up new possibilities for automating and improving attendance management. Web-based attendance systems offer several advantages over traditional methods, including increased accuracy, reduced administrative overhead, and improved accessibility. They can also provide real-time data and reporting capabilities, enabling organizations and educational institutions to make informed decisions about workforce management. (Othman, 2012). However, many existing web-based attendance systems still rely on manual input or require dedicated hardware, which can be inconvenient or expensive to implement. Moreover, they may not adequately address the challenges of verifying user presence.

2.2. GENERAL CONCEPTS

Understanding a blockchain-based attendance system, require one to understand the underlying concepts that constitute the technologies for the system.

2.2.1. Blockchain Technology

Blockchain Technology simply refers to an immutable transaction ledger, maintained within a distributed network of peer nodes each maintaining a copy of the ledger by applying transactions validated through a consensus protocol, grouped into blocks linked to one another using a hash signature

2.2.2. Geofencing Technology

Geofencing is a location-based technology that uses GPS, RFID, Wi-Fi, or cellular data to create virtual boundaries around a physical area, triggering actions when a device enters or exits that zone.

2.2.3. Attendance Management System

An Attendance Management System is a software solution designed to record, and manage the attendance of individuals such as employees or students, automatically and accurately.

2.2.4. Blockchain-Based Attendance Management System

A secure and tamper-proof solution that records attendance data on a decentralized ledger

2.2.5. Peer-To-Peer (P2p) System

A decentralized network architecture where each participant (peer) acts as both a client and a server, sharing resources directly without relying on a central server

2.2.6. Permissioned Ledger

A type of blockchain where only authorized participants can access, validate, or write data

2.2.7. Consensus Mechanism

A protocol used in blockchain systems to achieve agreement among distributed nodes on the validity of transactions and the state of the ledger

2.2.8. Smart Contracts

A self-executing program stored on a blockchain that automatically enforces and executes predefined rules or agreements when specific conditions are met, without the need for intermediaries.

2.2.9. Types Of Nodes

In a Hyperledger Fabric or blockchain network, nodes are categorized as:

- Peer Nodes: Host ledgers and smart contracts; execute and endorse transactions.
- Ordering Nodes (Orderers): Establish consensus and order transactions into blocks.
- Client Nodes: Submit transaction requests and interact with the network via SDKs.
- Anchor Peers: Facilitate communication between different organizations in a channel.

2.2.10. Channels In Hyperledger Fabric

Channels in Hyperledger Fabric provide a private communication pathway between a specific subset of network participants. Each channel maintains its ledger, which is only accessible to the members of that channel. This allows organizations to transact privately and securely without revealing sensitive information to the entire network. Since nodes can have multiple ledgers. Key aspects of channels include:

- Isolation: Transactions within a channel are isolated from those in other channels, ensuring privacy.
- Confidentiality: Only members of a channel can access the channel's ledger and transaction data.
- Governance: Channels can have their own policies and governance structures, allowing

for flexibility in managing permissions and roles.

2.2.11. Organizations In Hyperledger Fabric

Independent entities that own and manage peers, participate in governance, and have distinct identities via their Membership Service Provider (MSP). They collaborate within channels to share data securely.

2.3. RELATED WORKS

Attendance management systems have undergone significant evolution, transitioning from manual methods to advanced automated solutions that leverage modern technologies. This section provides a comprehensive review of existing attendance tracking systems, highlighting their methodologies, advantages, and limitations, and identifies how our geo-temporal attendance tracking portal addresses existing gaps.

1. Development of a Geo-Temporal Attendance Tracking Portal

This research paper presents the development of a geo-temporal attendance tracking portal designed to enhance traditional attendance management systems. The portal addresses the need for accurate and reliable attendance records by capturing both login/logout times and users' locations. Users interact with the system by entering their name and email, which triggers the recording of their entry time and geographical coordinates. The system is

equipped with a safe administrative interface, which is accessed using a username and password login. Authorized personnel can manage users' data, and enable the addition of new users to the database of the organization. (Harsh Chavda, March 2025)

2. **Implementation of a location-based service on the geolocation presence system web-based**

Integrating both temporal and spatial data offers a comprehensive approach to attendance management. Research has explored the use of geo-temporal data to enhance the accuracy and reliability of attendance systems. For instance, a study proposed a system that utilizes GPS data to confirm an employee's presence within a predefined geofence during working hours, thereby ensuring both time and location compliance.

However, challenges such as GPS inaccuracies in urban canyons, battery consumption, and privacy issues persist. Our proposed geo-temporal attendance tracking portal aims to address these challenges by combining GPS with Wi-Fi triangulation to improve indoor accuracy, implementing energy-efficient location tracking algorithms to minimize battery drain, and incorporating robust data encryption and privacy policies to protect user information. (F. N. Syamaidzar and J. Sutopo, 2023)

3. **GeoWorkTrack: A Comprehensive Real-Time GPS-Based Employee Monitoring and Attendance Management System**

This paper presents a state-of-the-art GPS-based system, GeoWorkTrack, for employee monitoring and attendance management. The system automates attendance tracking, enhances workforce oversight, and promotes productivity through the use of real-time location data and geofencing technologies. During office hours, the system monitors employee locations, logs entry and exit times, calculates attendance percentages, and detects overtime events. (Pankaj Tile, 2024)

LIMITATIONS

- Reduced GPS accuracy in some environments (example; indoors, tunnels).
- High battery consumption on continuous GPS tracking.

Geofencing Calculation: The system uses the **Haversine formula** or a point-in-polygon algorithm to determine if a user's coordinates are within the set geofence.

Formula:

$$d = 2r \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

where:

- d = distance between two points (user and geofence center)
- r = Earth's radius
- $\Delta\phi$ = difference in latitude
- $\Delta\lambda$ = difference in longitude

Figure 1: Haversine formula

Purpose: This helps to dynamically track users and update their geofence status in the database.

Percentage Calculation: To calculate the daily presence percentage based on time spent within the geofence boundaries relative to total office hours.

Bounding Box Coordinates: Used to calculate the bounding area for geofence monitoring.

Result: The results indicate the system's viability but highlight areas needing improvement, such as GPS optimization and battery consumption management. Further development could incorporate predictive algorithms for location data smoothing and low-power optimizations for continuous tracking

4. **The Development of a Mobile Application for a Reward-Based Student-Lecturer Appointment Using Ethereum Blockchain and Smart Contracts**

The primary objective of this project is to develop WeMeet Dapps, an application based on an object-oriented approach, utilizing blockchain and smart contracts, for rewarding student-lecturer appointments. The application is designed using Android Technology to assess the functionality of WeMeet Dapps. The scope of this project is focused on UTHM students, lecturers, and coordinators. Several modules are incorporated within the application, including the login module, view profile module, manage slot module, booking appointment module, manage booking appointment module, manage live chat module, manage attendance module, manage appointment history module, manage user module, and manage reward module. (Muhamad Syamim)

LIMITATIONS

- **Limited user scope:** The system is designed only for UTHM students, lecturers, and coordinators, which means it might not work well for other universities without major changes.
- **Dependence on Android:** Since it's built using Android technology, users with iPhones or other devices won't be able to use the app unless another version is developed.
- **Internet requirement:** The app likely needs a constant internet connection for blockchain interactions and live chats, which may be a problem in areas with poor connectivity.

5. Implementation of a web-based E-voting system using Hyperledger Fabric

Problem statement

Despite the growing interest in electronic voting (eVoting), current systems continue to face critical concerns around security, privacy, and public trust. Traditional voting methods are not only costly and inefficient but also struggle to meet the expectations of large, diverse electorates in a digital era. Similarly, existing eVoting platforms are vulnerable to cyberattacks and data breaches, which could compromise the integrity of election outcomes and expose sensitive voter information. (Ajinomisanghan, 2023).

A key challenge remains the preservation of voter anonymity (Ross Clarke, 2023) while ensuring each vote is verifiable and legitimate. Maintaining this balance is essential to protect voters from coercion or retaliation, while also ensuring the system's transparency and auditability. Without strong guarantees of confidentiality, accuracy, and fairness, public confidence in electronic elections is unlikely to grow. (Aneta Poniszewska-Marańda, 2022)

Proposed Solution

To address these challenges, this project proposes a blockchain-based voting system built on Hyperledger Fabric. By harnessing the decentralized, immutable, and permissioned architecture of Hyperledger, the system aims to provide a secure, web-based voting infrastructure that supports remote access and tamper-resistant vote recording. Voters will be able to cast their ballots securely from anywhere with internet access, enhancing participation while maintaining the integrity and confidentiality of their votes.

System constraints

Nevertheless, implementing such a system introduces complex design considerations, including privacy enforcement, scalability, and compliance with legal and regulatory frameworks. This research investigates how Hyperledger Fabric can overcome these barriers to deliver a trustworthy, modern eVoting platform, ultimately aiming to improve democratic processes through technology-driven transparency and reliability

2.4. PARTIAL CONCLUSION

The literature review reveals significant advancements in developing blockchain-based attendance systems, highlighting their potential to enhance security, transparency, and efficiency in elections. However, challenges such as scalability, usability, regulatory compliance, and public trust must be addressed. This study aims to build on the existing research by developing and evaluating a blockchain-based attendance system that addresses these challenges and contributes to the advancement of secure and transparent attendance clock-in processes.

CHAPTER 3: ANALYSIS AND DESIGN

3.1. INTRODUCTION

In this chapter, we explore the analysis and design of a smart attendance management system built for schools. The system combines geofencing technology, biometric verification, and blockchain (using Hyperledger Fabric) to improve how student attendance is recorded and managed.

We begin by clearly identifying the problems with current attendance tracking methods such as impersonation, inaccurate records, and manual work. We then present the methods used to solve these issues through a secure and automated approach.

This chapter follows the steps of the Software Development Life Cycle (SDLC), providing a clear and organized path from understanding the problem to designing a modern solution. The goal is to create a system that ensures only verified students in the right location at the right time can clock in, and that attendance data is securely stored, tamper-proof, and easy to audit using blockchain technology.

3.2. METHODOLOGY

This section outlines the project management approach used to develop the smart school attendance management system, which integrates geofencing, biometric verification, and blockchain technology (Hyperledger Fabric) to ensure secure, accurate, and verifiable attendance tracking. The development process followed the Agile Software Development Life Cycle (SDLC) model, leveraging an iterative and incremental approach to deliver working components in cycles known as sprints.

3.2.1. AGILE SDLC MODEL

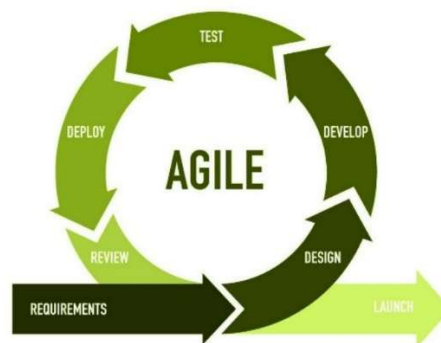


Figure 2: Agile Model

The Agile philosophy promotes the rapid development and deployment of modular components. For this project, the system was broken down into several independent services, enabling isolated development and testing of critical features without impacting the entire system.

The major modules included:

- Frontend (Mobile App and Admin web panel) for student interactions and admin duties
- Backend Server for business logic and blockchain interaction.
- Blockchain Network Layer using Hyperledger Fabric for decentralized attendance record keeping.

Why Agile?

Given the complex nature of integrating multiple technologies (mobile development, blockchain, biometric verification, and cloud services), the Agile methodology offered:

- Flexibility to iteratively integrate components like Google Auth, Fabric CA, and biometric
- Early delivery of functional features like login and clock-in and continuous user feedback
- Responsiveness to changing functional or academic policy requirements.

Values of Agile Applied to This Project

- **People Over Processes and Tools**

The system was designed with end-user experience in mind, specifically for students and lecturers. Their feedback helped shape intuitive workflows

- **Working Software Over Comprehensive Documentation**

Iteratively deployed features such as Google sign-in and geolocation verification helped the team validate design assumptions early, even as documentation evolved alongside.

- **Customer Collaboration Over Rigid Contracts**

Lecturers and administrators provided feedback through sprint reviews, influencing course/session configurations and data flow between mobile, backend, and blockchain layers.

- **Responding to Change Over Following a Plan**

The modular breakdown allowed flexibility in updating geofence logic, blockchain enrollments, or UI flows independently without affecting the whole system.

AGILE SDLC PHASES

3.2.2. REQUIREMENT ENGINEERING

What is Requirement Engineering?

A systematic and strict approach to the definition, creation, and verification of requirements for a software system. To guarantee the effective creation of a software product, the requirements engineering process entails several tasks that help in understanding, recording, and managing the demands of stakeholders. (community G. , 2025)

The Requirements Engineering Process

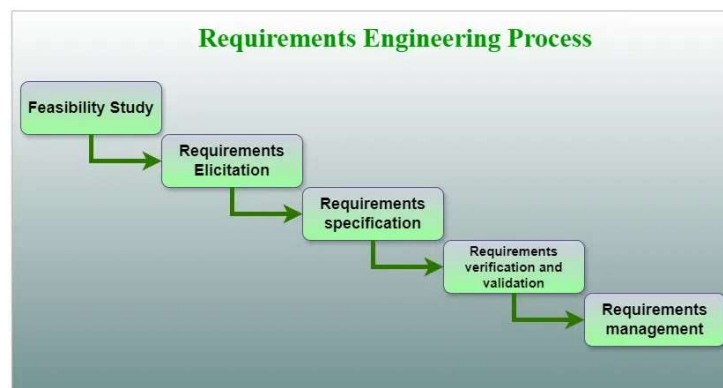


Figure 3: requirement engineering process

Feasibility Study

1. **Technical Feasibility**

The project is technically feasible. It will be developed using commonly available (Open Source) mobile technologies, such as GPS-enabled smartphones, and decentralized verification technologies (blockchain)

2. **Operational Feasibility**

The system addresses a real problem faced in an academic environment. It is designed to be user-friendly and easy to operate, even for users with limited digital skills.

3. Economic Feasibility

The initial cost of development is moderate, requiring mainly developer effort, access to hosting/cloud services, and mobile testing devices. The long-term benefits, such as saving time, reducing paperwork, and preventing fraud, outweighing the cost.

4. Legal Feasibility

The system complies with general data protection regulations by implementing user authentication and secure data handling. The system respects intellectual property by using open-source technologies or licensed tools where necessary.

5. Scheduled Feasibility

Tasks are broken down with achievable deadlines, and resources such as development tools, mentors, and devices are available on schedule to meet the project's goals.

Requirements Elicitation

Requirements elicitation is the process of gathering information about the needs and expectations of stakeholders for a software system. The goal of this step is to understand the problem that the software system is intended to solve and the needs and expectations of the stakeholders who will use the system. (Requirement Elicitation, 2025)

Requirements Gathering

Requirement gathering processes refer to the various steps and methods used to collect, understand, and document the needs and expectations of stakeholders regarding a project, product, or service. These processes are essential for clearly defining what needs to be accomplished and ensuring that the final outcomes meet the agreed-upon requirements and objectives. There are six crucial steps in requirement gathering



Figure 4: requirement gathering process

1. Assigning Roles

Table 1: requirement gathering roles assignment

User roles	Secondary Roles	Description
Students		▪ Checks in using geolocation and verification. ▪ Views personal attendance history
	Class Delegate	▪ Can escalate issues like system access problems or verification failures.
Lecturers		▪ Reviews and validates attendance data.
Admin		▪ Oversees overall attendance statistics. ▪ Handles dispute resolutions and attendance correction requests. ▪ Ensures the system aligns with school policy and compliance.

2. Define Project Scope

This study focuses on the development, testing, and evaluation of a mobile-based attendance system tailored for higher education institutions in Cameroon. It covers features such as student authentication, location verification through geofencing, real-time attendance recording, decentralized verification for data integrity, and generation of attendance reports. The scope is limited to the recording and management of class attendance only.

3. Conduct Stakeholder Interviews

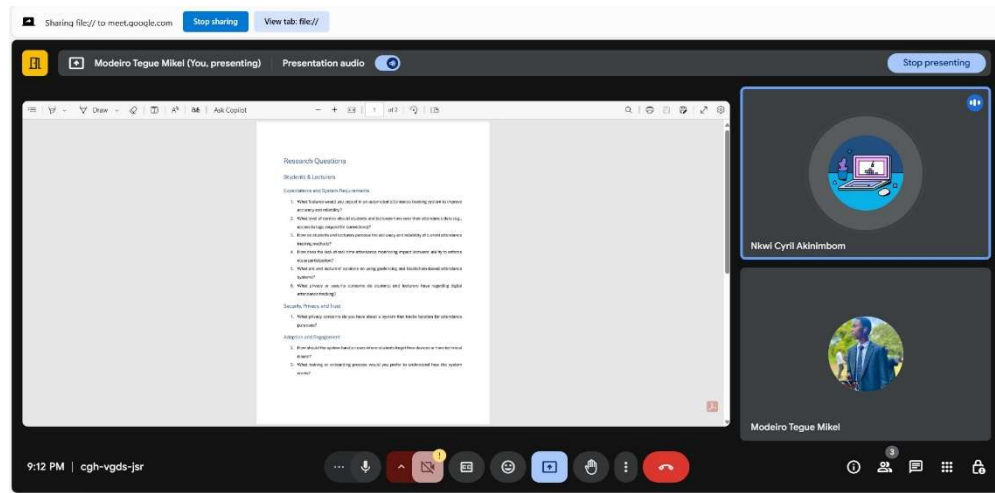


Figure 5: end-to-end interview 1

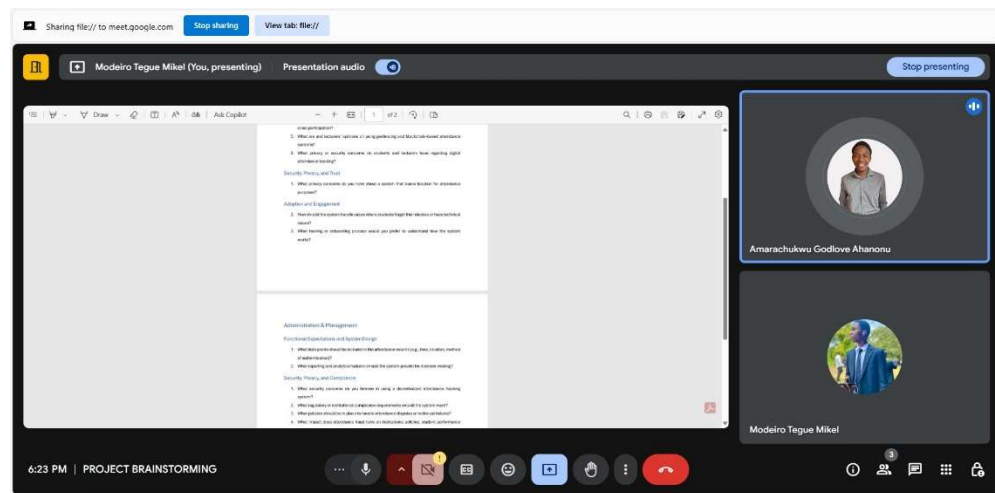
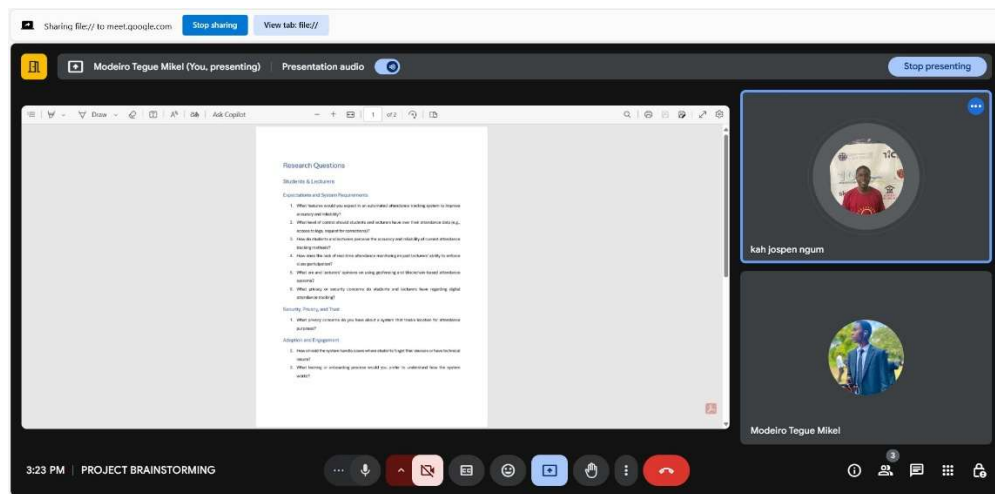


Figure 6: end-to-end Interview 2



```

INTERVIEW INFO
File Edit View

Cyril (class delegate)

Current system: Manual Paper record, excel sheet printed and
each student signs on his name, lecturers calling the name,
costly in printing, time consuming

drawbacks:
- Easy to tamper the attendance integrity,
- the attendance records could easily be destroyed
- could be stolen or get missing
- costly in printing, time consuming

Expectation:
- Secured
- Only lecturers should be able to get access to it
- students not in class shouldn't be able to record
- Processing time of attendance record
- Student Verification upon registration

Efficiency
Ln 66, Col 37 | 1,628 characters | 100% | Windows (CRLF) | UTF-8

```

```

INTERVIEW INFO
File Edit View

STEPHANE (student)

Instructor
initiate attendance system and send code for students to join
course (similar to google classroom flow)

2. CONTROL LEVEL
- Attendance history
- User profile

3. Criteria to evaluate Efficiency
- Accurate location monitoring and attendance tracking
- Attendance history

Course Management (attendance tracking per course)

Ln 42, Col 1 | 1,630 characters | 100% | Windows (CRLF) | UTF-8

```

Figure 7: end-to-end Interview 3

4. Document Requirement

The different stakeholder's demands where recorded and from there a google formed was used to draft out the overall outcomes as possible features, for the interviewed stakeholders (not limited to) to verify and approved the different features they ping-point during their end-to-end interview. (ref to link here)

Smart Attendance System – Stakeholder Requirements Survey

We are collecting input from students, lecturers, and administrative staff to design a mobile-based attendance tracking system that is secure, reliable, and tailored to the needs of our Institution.

This survey will help us understand your expectations, preferences, and concerns related to features such as geolocation tracking, course-based attendance sessions, and data access levels.

Your responses will directly influence the system's design and ensure it meets both academic and administrative requirements. The form is short and should take less than 5 minutes to complete.

Thank you for your valuable input!

Figure 8: requirement survey

5. Verify & Validate Requirements

The responses collected from the form served as the foundation for validating the accuracy and completeness of the system's requirements. This process helped in:

- Confirming which features were most relevant and expected by users.
- Identifying missing functionalities or misaligned assumptions.
- Prioritizing development efforts based on user-rated importance.

Verification ensures that the requirements reflect stakeholder intentions, while validation confirms they align with the project's goals, both carried out through static testing techniques like document reviews and stakeholder feedback before any software execution.

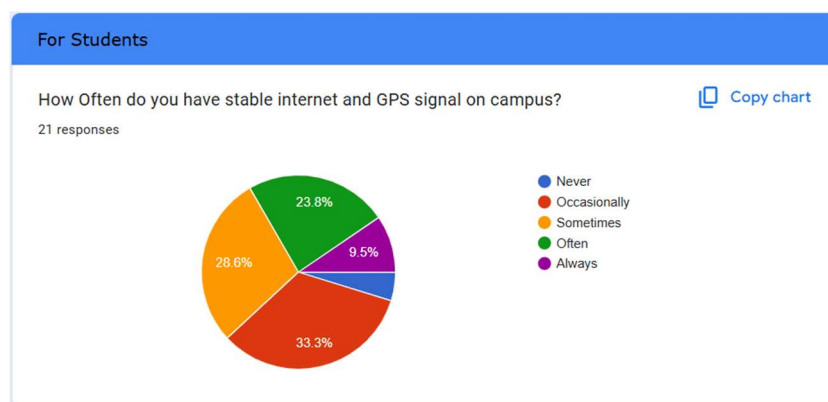


Figure 9: students survey stats

In some few words, on what would you base yourself to evaluate the efficiency of an attendance management system?
21 responses
Accuracy, real time monitoring
Open to potential miscount
Speed of taking, security in keeping attendance lists, error (a student might mark on someone else name)
Time management
the method and the accuracy of the collection of data
View attendance history per course
Speed and Accuracy of attendance marking
As for me, I would base the efficiency of attendance management system as to how fast it is capable of registering attendances and avoid duplication of attendance by students... Also take into consideration that one and only one student should be able to sign his attendance.

Figure 10: survey open answers

Requirements Analysis and Specification

Requirement analysis simply means complete study, analyzing, describing software requirements so that requirements that are genuine and needed can be fulfilled to solve problem. There are several activities involved in analyzing Software requirements as follows

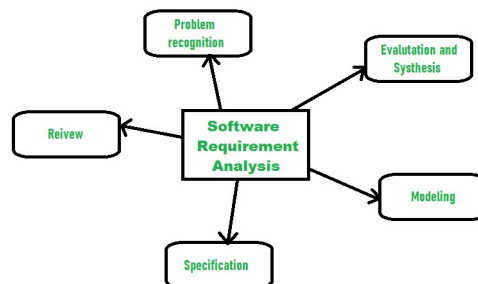


Figure 11: requirement analysis

1. Problem Recognition

Attendance management is a critical part of academic administration, yet many educational institutions in Cameroon still rely on outdated, manual methods. These traditional systems are prone to errors, manipulation, and inefficiencies, which affect student performance tracking and institutional reporting.

2. Evaluation & Synthesis

In requirement analysis, evaluation is the process of checking that gathered requirements are clear, complete, realistic, and aligned with project goals. It ensures they are feasible,

non-conflicting, and relevant. Synthesis follows by organizing and grouping these evaluated requirements into a logical, structured set. It prioritizes and connects related requirements to form a strong foundation for design, development, and testing stages

Below is a table to summarize the outcome of the evaluation phase. The constraints in the table above simply refers to a restriction or limitation that shapes how the requirement will be fulfilled

Table 2: requirement evaluation & synthesis

Requirements	Type	Constraints
Real-time notification	Non-functional (usability)	
Attendance Statistics	Functional	
GPS location verification	Non- functional	▪ Frequent GPS tracking can drain a device’s battery, which might be inconvenient to users. So, the verification will be performed only when attempting to mark attendance ▪ GPS accuracy must be within 10meters
Biometric Verification	Functional	Available biometric Hardware/Software?
Correction flow in case of attendance error	functional	
Course-level attendance tracking	Functional	
Metric for evaluating attendance efficiency	Non-functional (efficiency KPIs)	▪ Integrity of data ▪ Location accuracy ▪ Biometric verification
Attendance records	Functional	

Processing time to complete an attendance marking (quick check-in)	Non-functional	
Avoiding duplicate check-ins	Non-functional	
System initiation	Admin initiates system	
Decentralization verification	Non-functional	

3. Modeling

A Complete Modeling description is provided in the system design section

4. Specification

Specification is an activity carried out that helped me to come out with a very important and crucial document called the Software Requirement Specification (SRS).

3.2.3. SRS DOCUMENT

INTRODUCTION

I. PURPOSE OF THIS DOCUMENT

This document defines the Software Requirements Specification (SRS) for a mobile-based School attendance management system, designed for higher education institutions in Cameroon. The purpose is to outline the functional and non-functional requirements, user interactions, and constraints that will guide the development of the system

II. SCOPE OF THIS DOCUMENT

The system aims to automate student attendance geolocation verification, and decentralized validation, for secure and immutable records. The document details the core features and outlines the development scope. The system focuses solely on classroom attendance tracking.

III. OVERVIEW

This system offers a solution to the current manual attendance process, which is time-consuming and prone to fraud. With geofencing capabilities, the application provides a real-time, secure, and reliable method for recording attendance. It includes role-based access for students, lecturers, and administrators, and generates attendance reports.

GENERAL DESCRIPTION

The mobile application allows students to register their attendance and ensures they are within a specified geofenced location. The application supports course level attendance management, and provides detailed reports to users. It aims to improve accuracy, transparency, and data integrity in the attendance process.

- **User Goals:** Record attendance accurately and conveniently.
- **User Characteristics:** Students, lecturers, and admins familiar with mobile apps.
- **Key Benefits:** Time-saving, secure, and reliable attendance tracking.
- **Importance:** Enhances accountability, minimizes impersonation and errors.

FUNCTIONAL REQUIREMENTS

- Attendance Tracking Mechanism
- Access to attendance statistics
- Course and Session management
- Attendance Reporting

NON-FUNCTIONAL REQUIREMENTS

- **Usability**
Many users in Cameroon may not be very tech-oriented, so the app must have a clean, simple interface that works well on commonly used smart phones, and doesn't require much training.
- **Data integrity**
When a student is marked present or absent, the system must ensure this record is accurate and can't be changed by unauthorized people. This helps avoid cheating or mistakes in attendance.
- **Decentralized verification**
Instead of one person or server controlling attendance records, multiple nodes can help confirm the data, making it harder to fake attendance and easier to build trust in the records.
- **Compliance**

In Cameroon, the system must respect the Personal Data Protection Act, especially when collecting biometric data like fingerprints. It should also borrow good practices from laws like the GDPR to protect users' privacy.

- **Reliability & Availability**

Classes in Cameroon happen on a fixed schedule. So, the system must be available every day with minimal downtime, even when internet or power is unstable. It should store data offline and sync when back online if needed.

- **Portability**

Whether on different phones, schools, or institutions in Cameroon, the app should work without needing a lot of technical support. This allows schools with limited resources to use it easily.

- **Scalability**

As more schools and universities in Cameroon adopt the system, it should still perform well without slowing down or needing a complete rebuild. It should be able to handle more users, courses, and attendance records over time

INTERFACE REQUIREMENTS

- User-friendly login screen with Google OAuth support
- Intuitive dashboard for attendance monitoring
- Biometric authentication prompt for clock-in
- Excel upload option for initial data import
- Editable course and session management panel
- Error notification system with clear messages

PERFORMANCE REQUIREMENTS

Static: The system must process and record attendance for up to 500 concurrent users within 5 seconds each with 90% accuracy, ensuring efficient performance under peak load

Dynamic: The geofencing accuracy should be within a 10-meter and attendance should be recorded in less than 10 seconds per student

3.3. SYSTEM DESIGN

This chapter outlines the design framework of the blockchain-based attendance management system. It defines the architectural structure, behavioral logic, and interface interactions that guide the system's development. The design process serves as a blueprint that transforms the system's functional requirements into actionable technical solutions.

To support this, various UML diagrams are used. Structural diagrams like class diagrams to represent the static architecture. Behavioral diagrams such as use case, sequence, activity, and state diagrams illustrate the system's dynamic behavior and user interactions.

To ensure clarity and organization, the design is presented in three main sections:

- High-Level Design (HLD)
- Low-Level Design (LLD)

3.3.1. HIGH-LEVEL DESIGN

HLD plays a significant role in developing scalable applications, as well as proper planning and organization. High-level design serves as the blueprint for the system's architecture, providing a comprehensive view of how components interact and function together. This high-level perspective is important for guiding the detailed implementation phase and ensures scalability, maintainability, and performance in large-scale applications. (GeeksforGeeks, n.d.)

Components of High-Level Design

Understanding the components of high-level design is very important for creating effective systems that meets user needs and technical requirements. Below are the main components of high-level design

System Global Architecture

The system follows a modular, layered architecture combining off-chain and on-chain components:

- Frontend Layer:
 - Expo Mobile App for students handles Google sign-in, biometric verification, geolocation, and clock-ins.

- React Web Panel for admin, enables Excel uploads and session setup.
- Backend Server (Node.js):
Handles Excel parsing, data management, Fabric CA enrollment, and transaction signing via the Fabric SDK.
- Blockchain Layer (Hyperledger Fabric):
Ensures secure, tamper-proof attendance records, manages user identity via certificates, and validates transactions.
- Storage Layer:
 - Off-chain database stores user profiles and attendance metadata.

This design ensures decentralization, security, and real-time attendance validation. (see Figure 22: Global system architecture)

UML SEQUENCE DIAGRAM

1. User authentication

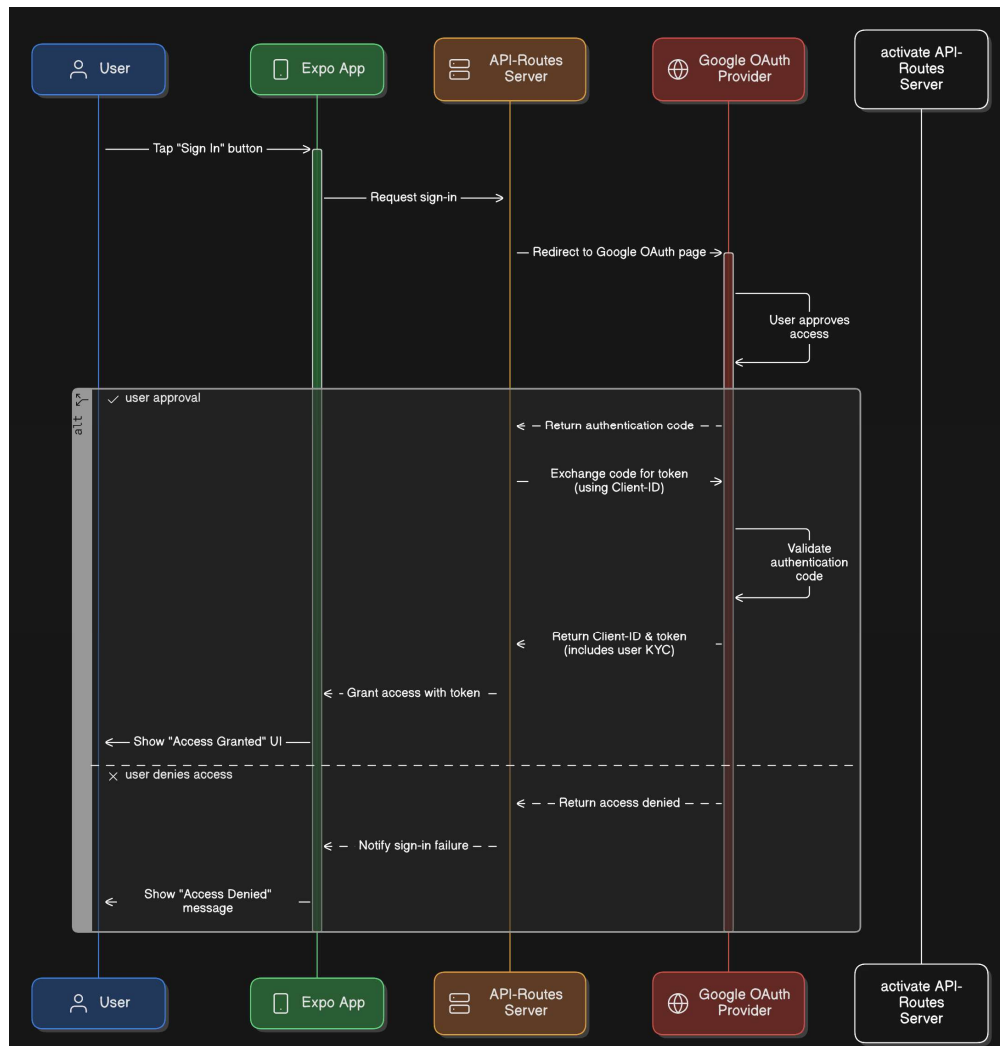


Figure 12: sequence of user authentication

Description

This sequence diagram illustrates the authentication flow the Expo App using Google OAuth for user sign-in.

- Initiation:** The User taps the "Sign In" button in the Expo App.
- Request:** The Expo App sends a sign-in request to the API-Routes Server.
- Redirect:** The API-Routes Server redirects the User to the Google OAuth page.
- Approval:** The User approves or denies access on the Google OAuth page.

- **If Approved:** Google OAuth returns an authentication code to the API-Routes Server.
 - **If Denied:** The process ends, and the User sees an "Access Denied" message.
- e. **Token Exchange:** The API-Routes Server exchanges the authentication code for a token using the Client ID.
- f. **Validation:** The activate API-Routes Server validates the authentication code.
- g. **Access Grant:** If valid, the API-Routes Server returns a client ID and token (including user KYC) to the Expo App, granting access with the token.
- h. **Notification:** The Expo App notifies the User of success ("Access Granted") or failure ("Sign-in Failure" if access is denied).

2. On-Chain Enrollment and Attendance Clock-in

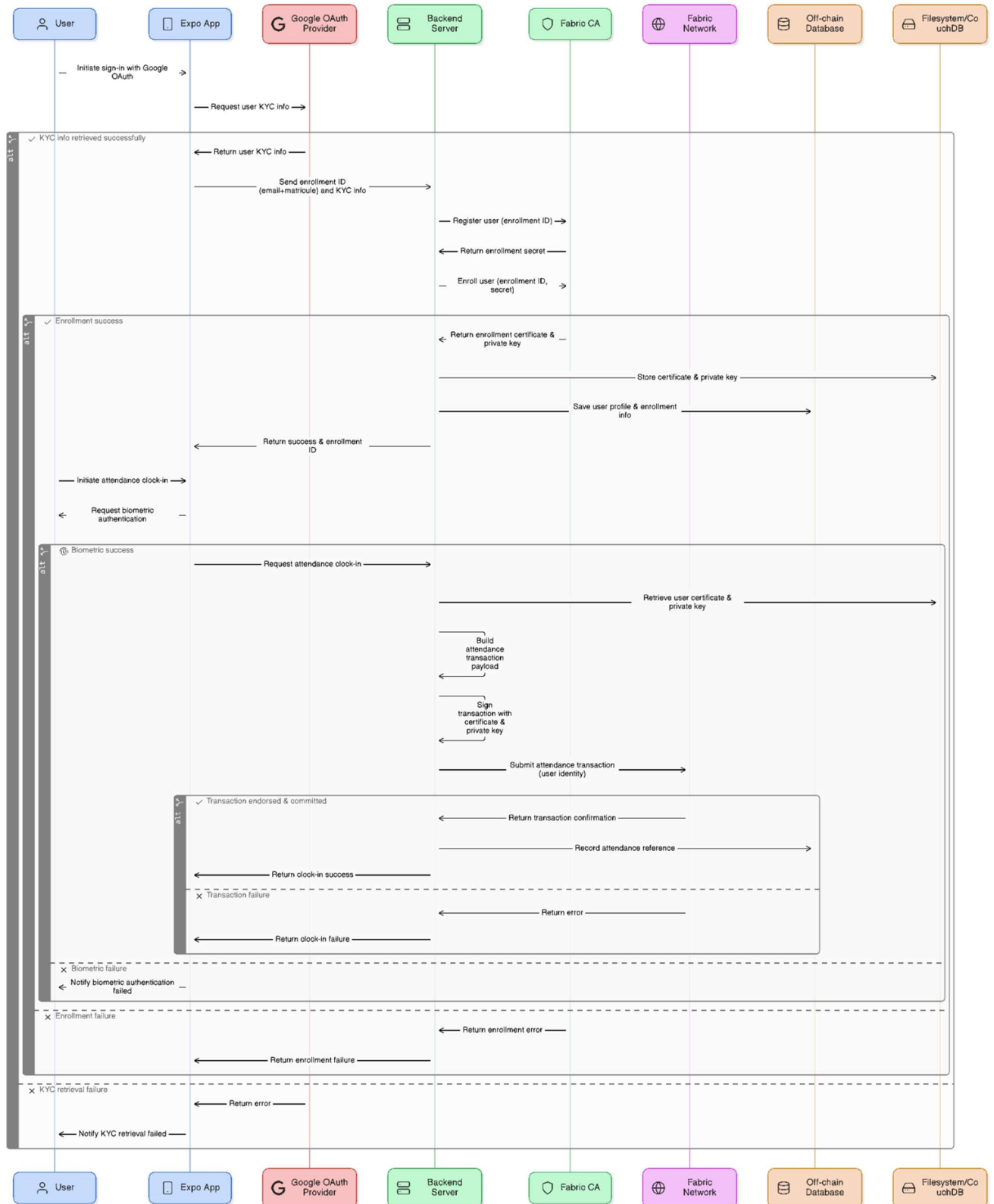


Figure 13: sequence of on-chain enrollment and clock-in

This sequence diagram illustrates the process of user authentication, enrollment, and attendance tracking using the Expo App, integrated with Google OAuth, a Backend Server, Fabric CA, Fabric Network, Off-Chain Database, and Filesystem/CouchDB. explanation details are as follows:

a. Initiate Sign-In with Google:

- The User initiates a sign-in via the Expo App, requesting KYC (Know Your Customer) information.
- The Expo App sends this request to the Google OAuth Provider.
- If successful, the Google OAuth Provider returns user KYC info (e.g., email, name, and KYC ID) to the Expo App.

b. User Enrollment:

- The Expo App forwards the KYC info and enrollment ID to the Backend Server.
- The Backend Server registers the user (enrollment ID) with Fabric CA.
- Fabric CA returns an enrollment secret, and the Backend Server enrolls the user, receiving a certificate and private key.
- The Backend Server stores the certificate and private key, saves the user profile and enrollment info to the Off-Chain Database, and returns enrollment success to the Expo App.
- If enrollment fails, an error is returned.

c. Initiate Attendance Check-In:

- The User initiates an attendance check-in via the Expo App.
- The Expo App requests biometric authentication from the User.
- If successful, the Expo App sends the attendance check-in request (including user identity) to the Backend Server.

d. Attendance Transaction:

- The Backend Server retrieves the user certificate and private key from the Filesystem/CouchDB.

- It builds a transaction (attendance payload), signs it with the private key, and submits the attendance transaction to the Fabric Network.
- The Fabric Network endorses and commits the transaction, returning transaction confirmation.
- The Backend Server records the attendance reference in the Off-Chain Database and returns check-in success to the Expo App.
- If the transaction fails, an error is returned.

User Roles and System Sub-modules

To clearly define how users interact with the system, we categorize the application into distinct user roles and functional modules. Each role is associated with specific actions within the system:

UML USECASE DIAGRAMS

1. Student Use Cases

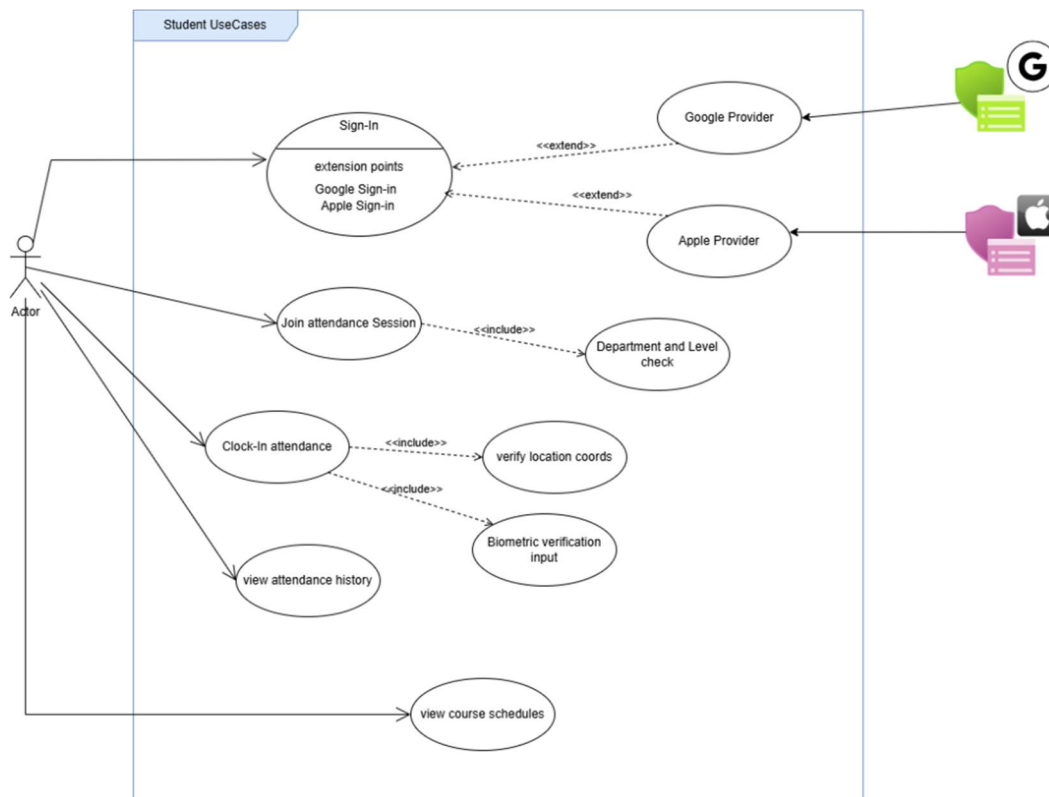


Figure 14: student use cases

Description

Components

- **Actor:** The student, who initiates all use cases.
- **Use Cases:** Specific functionalities the system provides.
- **Providers:** Google Provider and Apple Provider handle authentication.
- **Relationships:** Include, extend, and association links define how use cases interact.

Use Cases

1. Sign-In:

- **Description:** The primary process for the student to log into the system.
- **Extensions:**
 - **Google Sign-In:** Authenticates using Google Provider.
 - **Apple Sign-In:** Authenticates using Apple Provider.
- **Relationship:** Both sign-in methods extend the main Sign-In use case, offering alternative authentication options.

2. Join Attendance Session:

- **Description:** Allows the student to participate in an attendance session.
- **Includes:**
 - **Department and Level Check:** Verifies the student's department and level before joining.
- **Relationship:** This is a prerequisite step included in the join process.

3. Clock-In Attendance:

- **Description:** Records the student's attendance.
- **Includes:**
 - **Verify Location Coords:** Ensures the student is at the correct location.

- **Biometric Verification Input:** Uses biometric data (e.g., fingerprint) to confirm identity.
- **Relationship:** Both verification steps are essential parts of the clock-in process.

4. **View Attendance History:**

- **Description:** Allows the student to review their past attendance records.
- **Relationship:** A standalone use case for personal tracking.

5. **View Course Schedules:**

- **Description:** Enables the student to see their course timetable.
- **Relationship:** A standalone use case for planning purposes.

Relationships:

- **Include:** Indicates that one use case (e.g., Clock-In Attendance) incorporates the functionality of another (e.g., Verify Location Coords, Biometric Verification).
- **Extend:** Shows that Google Sign-In and Apple Sign-In are optional extensions of the Sign-In use case.
- **Association:** Links the Student Actor to all use cases, indicating their initiation of these actions.

External Systems:

- **Google Provider:** Handles Google Sign-In authentication.
- **Apple Provider:** Handles Apple Sign-In authentication.

2. Instructor Use Cases

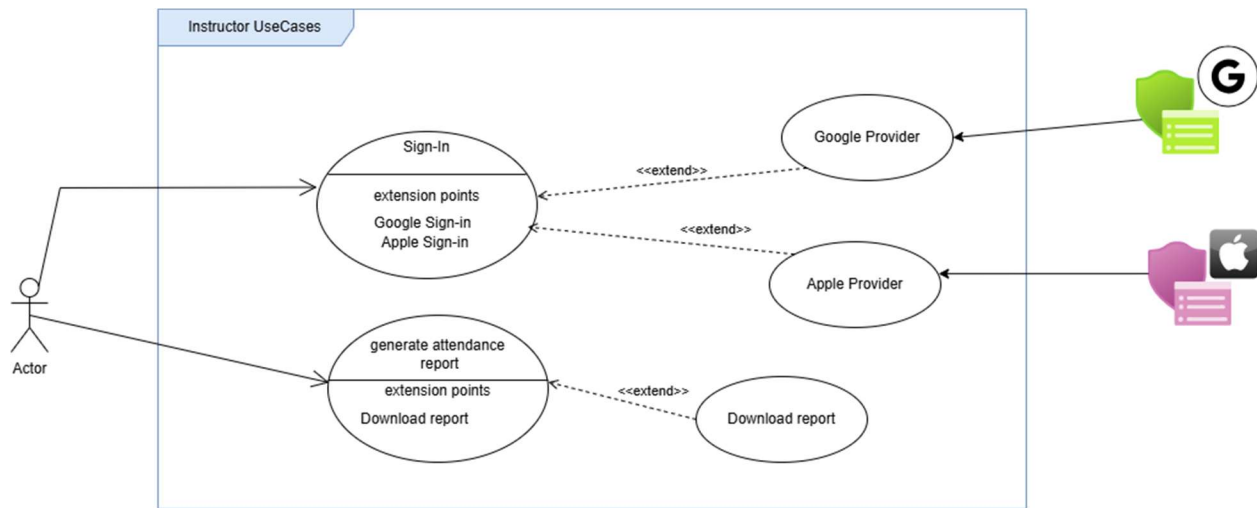


Figure 15: instructor use cases

Components:

- **Actor:** The instructor, who initiates all use cases.
- **Use Cases:** Specific functionalities the system provides.
- **Providers:** Google Provider and Apple Provider handle authentication.
- **Relationships:** Include, extend, and association links define how use cases interact.

Use Cases:

1. Sign-In:

- **Description:** The primary process for the instructor to log into the system.
- **Extensions:**
 - **Google Sign-In:** Authenticates using Google Provider.
 - **Apple Sign-In:** Authenticates using Apple Provider.
- **Relationship:** Both sign-in methods extend the main Sign-In use case, offering alternative authentication options.

2. Generate Attendance Report:

- **Description:** Allows the instructor to create an attendance report.

- **Extension Point:**
 - **Download Report:** Provides the option to download the generated report.
- **Relationship:** Download Report extends the Generate Attendance Report use case, adding an optional action.

3. Download Report:

- **Description:** Enables the instructor to download the attendance report.
- **Relationship:** This use case is an extension of Generate Attendance Report, triggered as needed.

Relationships:

- **Extend:** Indicates that Google Sign-In and Apple Sign-In are optional extensions of the Sign-In use case. Similarly, Download Report extends Generate Attendance Report.
- **Association:** Links the Instructor Actor to all use cases, indicating their initiation of these actions.

External Systems:

- **Google Provider:** Handles Google Sign-In authentication, represented with a Google icon.
- **Apple Provider:** Handles Apple Sign-In authentication, represented with an Apple icon.

3. System Auto-Use Cases

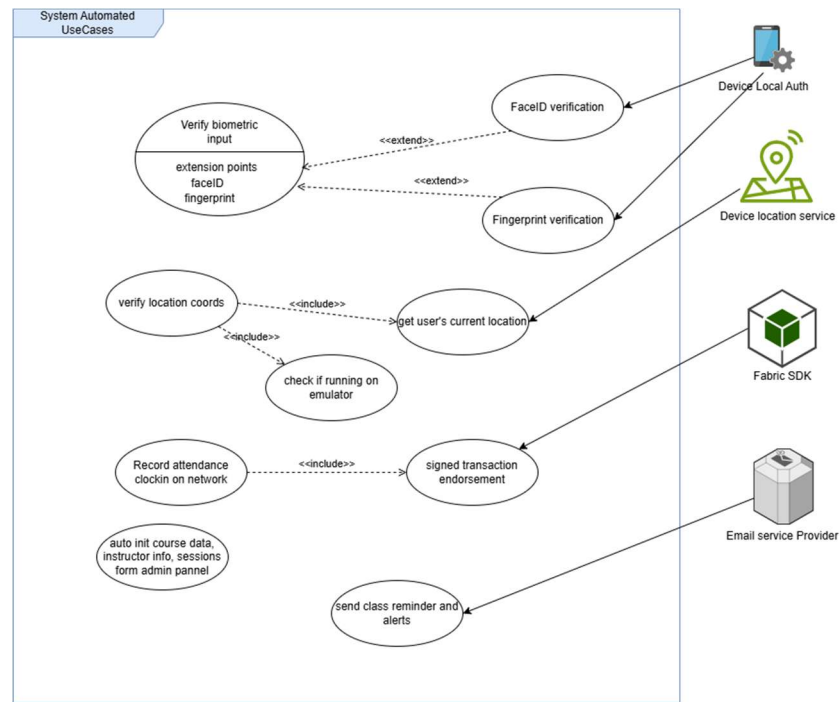


Figure 16: system auto use cases

Description

Components

- **Use Cases:** Automated processes within the system.
- **External Systems:** Device Local Auth, Device Location Service, Fabric SDK, and Email Service Provider support the use cases.
- **Relationships:** Include and extend links define how use cases interact.

Use Cases

a. Verify Biometric Input:

- **Description:** Verifies the user's biometric data for authentication.
- **Extension Points:**
 - **Face-ID Verification:** Uses facial recognition (extends the main use case).
 - **Fingerprint Verification:** Uses fingerprint scanning (extends the main use case).

- **Relationship:** Both verification methods are optional extensions of the biometric input process, supported by Device Local Auth.

b. **Verify Location Coords:**

- **Description:** Confirms the user's location coordinates.
- **Includes:**
 - **Get User's Current Location:** Retrieves the user's real-time location.
 - **Check if Running on Emulator:** Ensures the process isn't simulated.
- **Relationship:** These steps are essential parts of location verification, supported by Device Location Service.

c. **Record Attendance Clock-In on Network:**

- **Description:** Logs the attendance check-in on a network.
- **Includes:**
 - **Signed Transaction Endorsement:** Ensures the transaction is validated and endorsed.
- **Relationship:** Endorsement is a critical step, supported by Fabric SDK for blockchain-based validation.

d. **Auto Init Course Data, Instructor Info, Sessions from Admin Panel:**

- **Description:** Automatically initializes course data, instructor information, and session details from an admin panel.
- **Relationship:** A standalone use case for setting up system data.

e. **Send Class Reminder and Alerts:**

- **Description:** Sends reminders and alerts to users.
- **Relationship:** A standalone use case, supported by Email Service Provider.

Relationships:

- **Include:** Indicates that Verify Location Coords includes getting the current location and checking for emulators. Record Attendance includes signed transaction endorsement.
- **Extend:** Shows that Face-ID Verification and Fingerprint Verification are optional extensions of Verify Biometric Input.
- **Association:** Links external systems (Device Local Auth, Device Location Service, Fabric SDK, Email Service Provider) to their respective use cases.

External Systems:

- **Device Local Auth:** Supports biometric verification (Face-ID and Fingerprint).
- **Device Location Service:** Provides location data for verification.
- **Fabric SDK:** Manages blockchain-based transaction endorsement.
- **Email Service Provider:** Handles sending reminders and alerts.

3.3.2. LOW-LEVEL DESIGN

Low-Level Design (LLD) is the detailed design process in the software development process that focuses on implementing individual components described in the High-Level Design. (Low Level Design, 2025). It Breaks down each component from the high-level overview into internal modules and logic flows, describing data models, method interactions, and internal workflows using detailed class diagram, state machine Diagram, and activity diagram, ER-diagram and relational Model

UML DIAGRAMS

1. Class Diagram

Nature of relation-ships between classes

- A Student has one Attendance.
- A Session destroys one Attendance within it.
- A Course has many Sessions.
- An instructor teaches one Course.
- A Venue has a Geofence Validator.
- A Device implements Biometric AUTH.
- Blockchain Adapter depends on an adapter to enroll a transaction to the network.

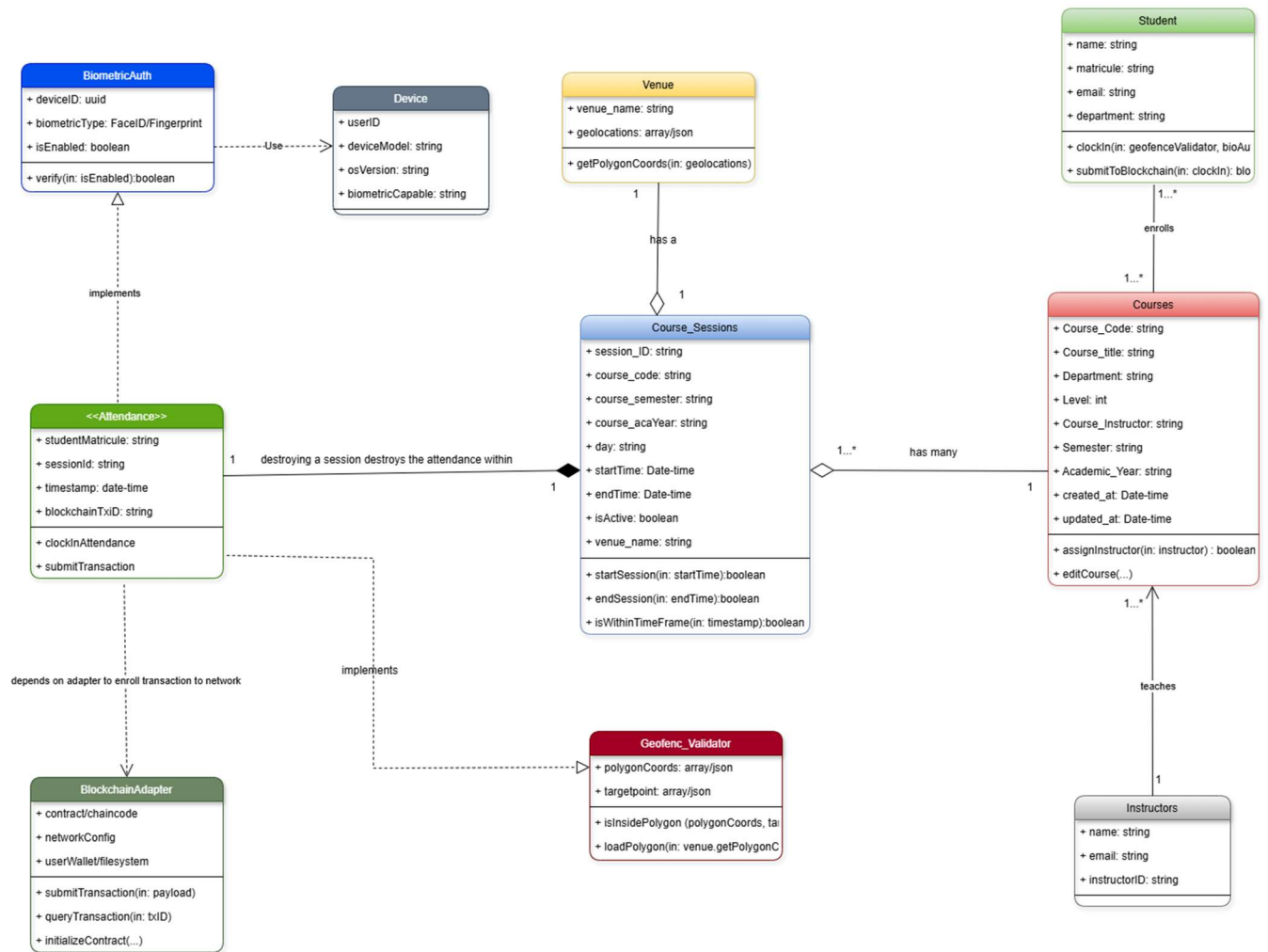


Figure 17: class diagram

2. State Machine Diagram

Purpose:

Models the different states an object goes through in its lifetime, and how it transitions between those states based on events or conditions.

Description

- [Start] → Logged Out
- Logged Out → Authenticating: Enter Matricule + dept + level
- Authenticating → Browsing Courses: Auth success
- Authenticating → Logged Out: Auth failed

- Browsing Courses → Joining Session: Join a course
- Joining Session → Verifying Location: Session active
- Joining Session → Browsing Courses: No session / Retry
- Verifying Location → Verifying Biometric: Inside geofence
- Verifying Location → Joining Session: Outside geofence
- Verifying Biometric → Submitting to Blockchain: Face match
- Verifying Biometric → Joining Session: Face mismatch
- Submitting to Blockchain → Clock-In Successful: TX confirmed
- Submitting to Blockchain → Clock-In Failed: TX rejected / error
- Clock-In Failed → Joining Session: Retry
- Clock-In Successful → [End]

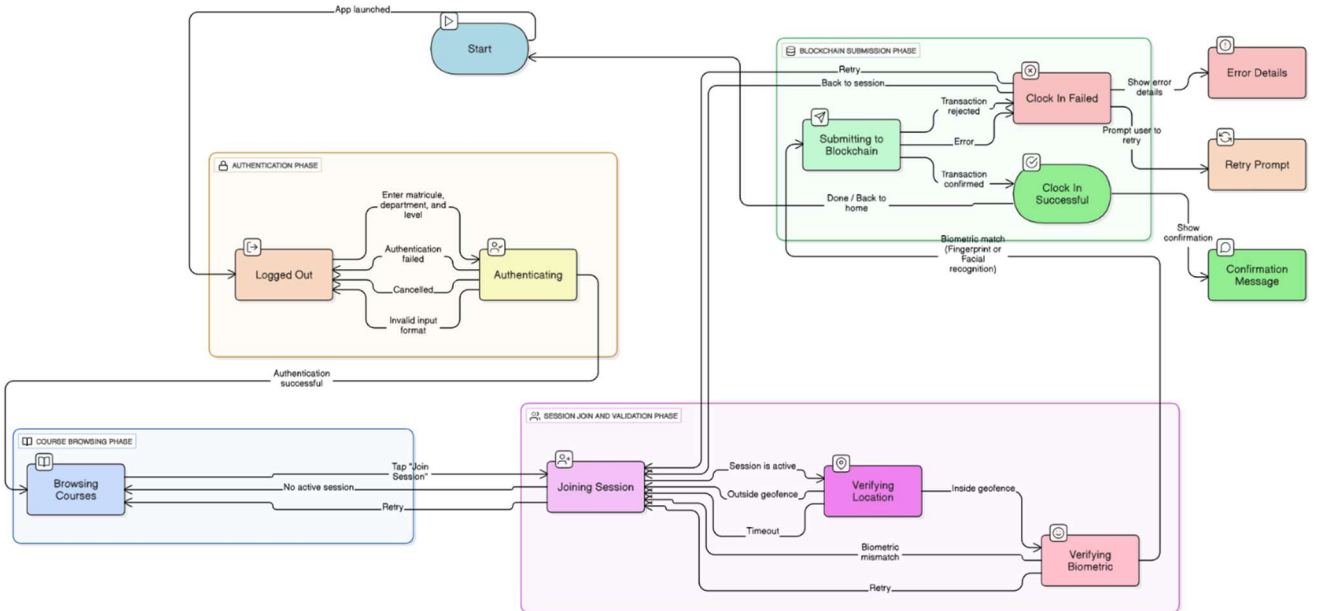


Figure 18: state machine diagram

3. Activity Diagram

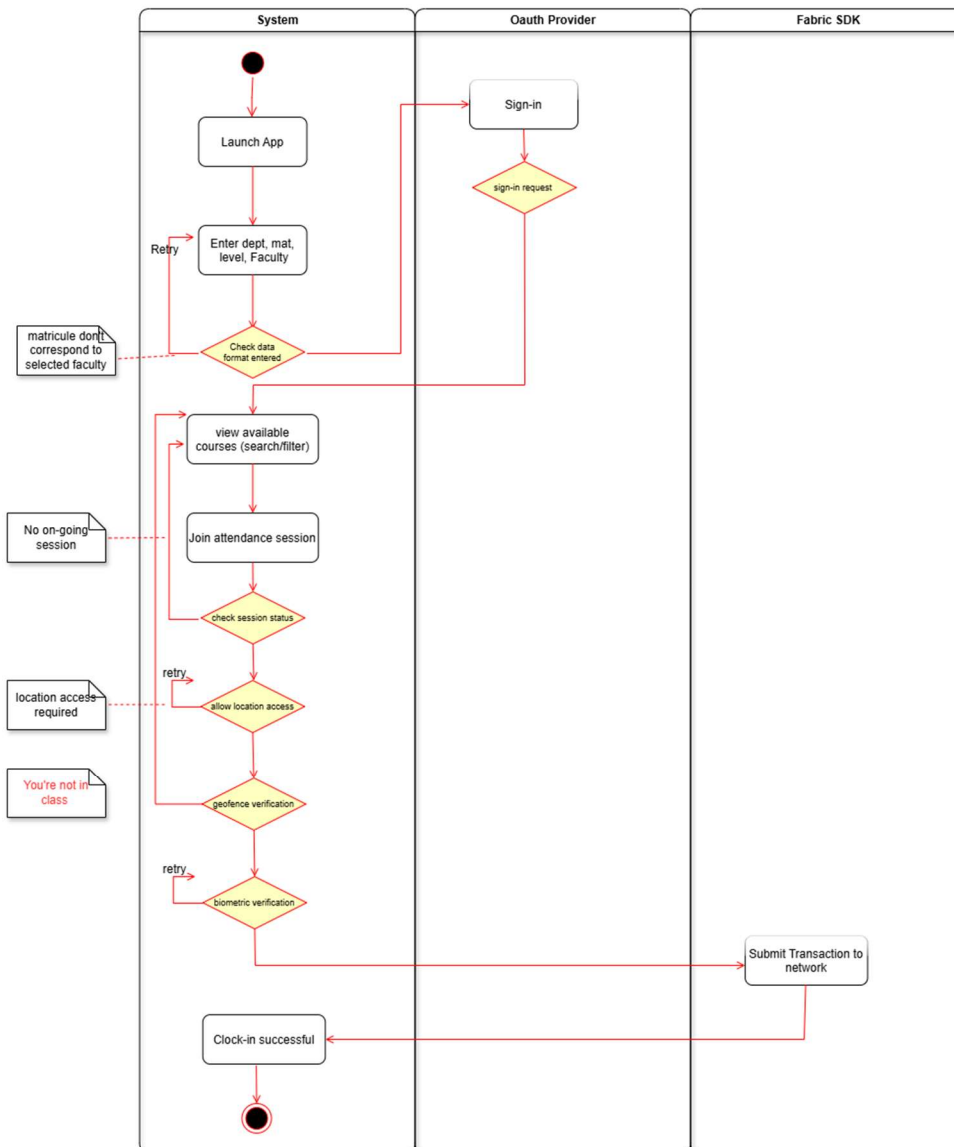


Figure 19: activity diagram

Description

This activity diagram outlines the attendance check-in process for a system integrated with OAuth Provider and Fabric SDK. The flow begins with launching the app, entering department, Matricule, level, and faculty details, followed by a sign-in request to the OAuth Provider. It checks the entered data format, then allows the user to view available courses (search/filter). The user joins an attendance session, checking session status, requesting location access, verifying geofence (ensuring the user is in class), and performing biometric verification. Upon successful completion of these steps, the system submits a transaction to the Fabric SDK for blockchain validation,

culminating in a successful clock-in. Failure at any decision point, for instance; no ongoing session, location access denied, not in class, or biometric failure, triggers a retry or ends the process with an appropriate message.

4. ER-Diagram and Relational Model

The diagram below illustrates the entity relation-ship of the different off chain entities of the system along with their respective attributes and cardinalities

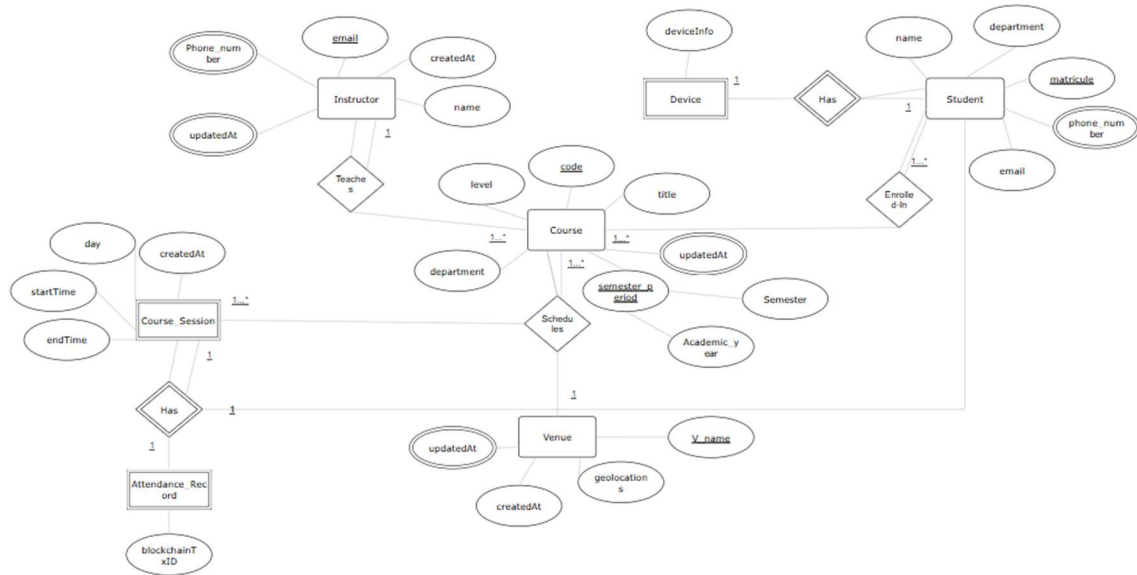


Figure 20: ER-Diagram

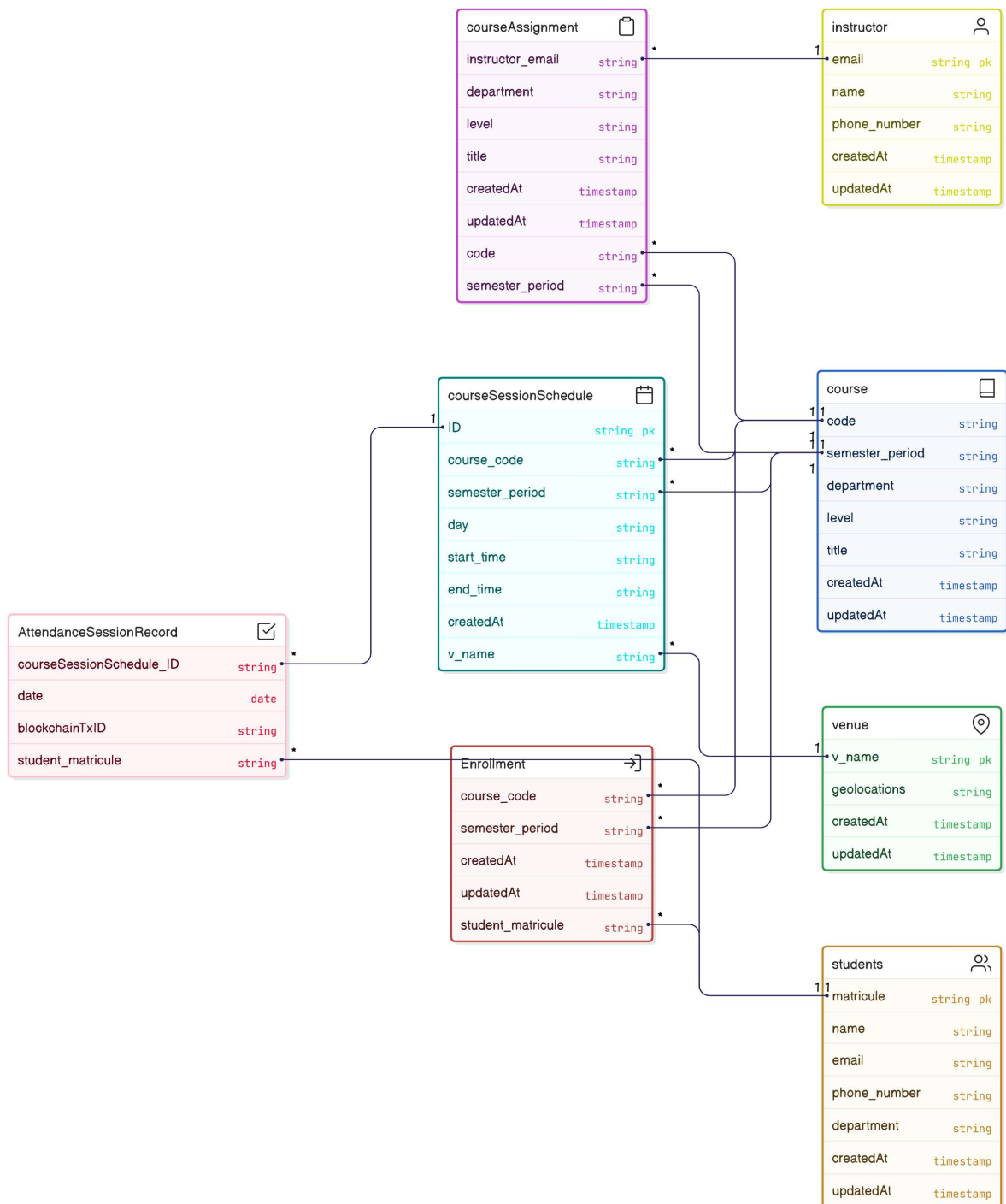


Figure 21: Relational model (schema)

3.4. GLOBAL ARCHITECTURE OF THE SOLUTION

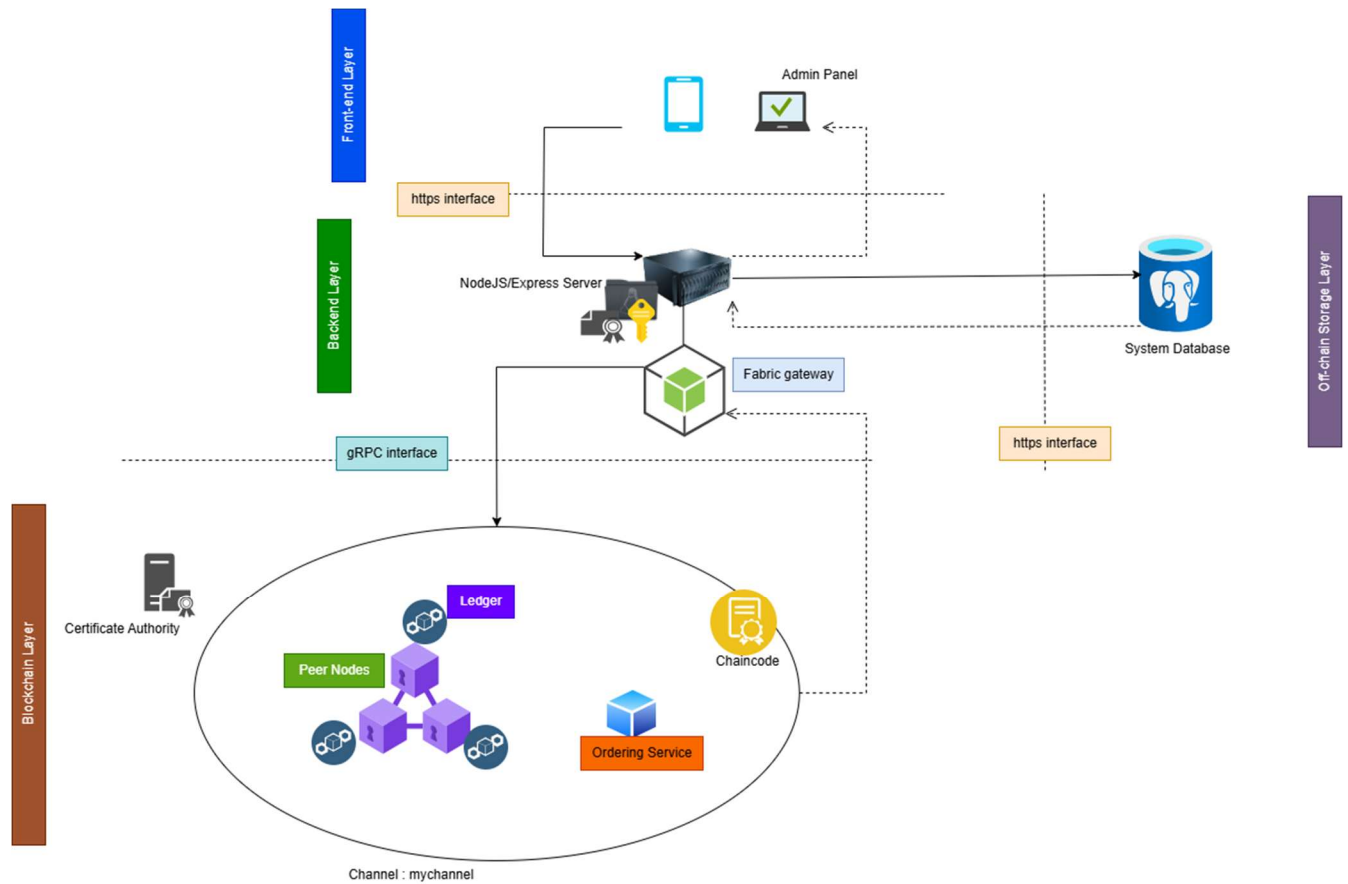


Figure 22: Global System Architecture

3.5. DESCRIPTION OF THE ALGORITHMS

This section describes the key logic or algorithms that power the system. This includes;

Ray Casting Algorithm for Geofencing

The ray casting algorithm is a simple and effective method used to determine whether a point, such as a student's location, lies inside a geofenced area, such as a classroom boundary. In the context of the Smart Attendance Management system, it is adapted to check if a student's coordinates fall within a predefined polygon defined by latitude and longitude points, ensuring accurate attendance validation.

Working Mechanism

The algorithm operates by casting an imaginary ray from the student's coordinates in a horizontal direction (typically to the right) and counting the number of times it intersects the edges of the geofence polygon. The geofence is represented as a closed shape with a series of coordinate pairs.

The process involves:

- i. **Ray Definition:** A ray is drawn from the student's latitude-longitude position horizontally.
- ii. **Intersection Counting:** The algorithm checks each edge of the polygon to see if the ray crosses it, considering only edges that span the ray's path.
- iii. **Decision Rule:** If the number of intersections is odd, the student is inside the geofence; if even, the student is outside. This leverages the fact that a point inside a polygon will cross an odd number of boundaries.

Formula or Mathematical Expression for the Check

The check relies on a geometric intersection test between the ray and each polygon edge. For an edge defined by points $(X1, Y1)$ and $(X2, Y2)$ and a ray from the student's point (x, y) horizontally to the right:

- **Intersection Condition:** The ray intersects the edge if:
 - The ray's y-coordinate y lies between $Y1$ and $Y2$ or $(Y2$ and $Y1)$
 - The x-coordinate of the intersection $X_{int} = X1 + \frac{(y - Y1)(X2 - X1)}{Y2 - Y1}$ is greater than X , ensuring the crossing is to the right.
- **Count Rule:** Increment the intersection count if the above conditions are met, and use the parity (odd/even) to determine inside/outside status.

```

Function IsInsideGeofence(studentX, studentY, polygon):
    Initialize crossings = 0

    For i from 0 to polygon.length - 1:
        point1 = polygon[i]
        point2 = polygon[(i + 1) % polygon.length]

        // Check if ray intersects the edge
        if (studentY > min(point1[1], point2[1]) and
            studentY <= max(point1[1], point2[1]) and
            studentX <= max(point1[0], point2[0])):
            if (point1[1] != point2[1]): // Avoid vertical edges
                xIntersect = point1[0] + (studentY - point1[1]) * (point2[0] - point1[0]) / (point2[1]
- point1[1])
                if (studentX < xIntersect):
                    crossings = crossings + 1

    if crossings % 2 == 1:
        Return "Inside Geofence"
    Else:
        Return "Outside Geofence"

```

Figure 23: Ray casting Pseudo code

Chain-code Algorithm

The Attendance Chain-code extends the Contract class from the fabric-contract-api to define smart contract functions that interact with the blockchain ledger. The primary function, clockIn, records a student's attendance by creating an immutable entry linked to a session and student ID, including a timestamp and transaction ID. It also emits an event (AttendanceSubmitted) for real-time updates, ensuring transparency and auditability

```

Class AttendanceContract:
    Function clockIn(context, sessionID, studentID, Name):
        // Get current timestamp from transaction
        timestamp = ConvertTransactionTimestampToISO(context.getTxTimestamp())

        // Create attendance record with provided and generated data
        attendanceRecord = {
            sessionID: sessionID,
            studentID: studentID,
            Name: Name,
            timestamp: timestamp,
            txId: context.getTxID()
        }

        // Store record on the ledger with a composite key
        key = sessionID + ":" + studentID
        StoreState(key, ConvertToBuffer(attendanceRecord))

        // Emit event for real-time notification
        EmitEvent("AttendanceSubmitted", ConvertToBuffer(attendanceRecord))

        // Return the record as a string
        Return ConvertToString(attendanceRecord)

```

Figure 24: Chain-code pseudo code

3.6. DESCRIPTION OF THE RESOLUTION PROCESS

How MarkX Solves Project Goals?

MarkX addresses the initial pain points of fraud, inaccuracy, and time inefficiency in traditional paper-based attendance systems through innovative technologies.

- **Geofencing Prevents Remote/Fake Check-ins:** geofencing ensures attendance is only recorded when a student is physically present, blocking remote or fake check-ins. This tackles the pain point of fraudulent attendance claims from outside locations.
- **Biometric Verification Ensures Real Student Check-ins:** Integrating Expo Local Authentication (e.g., fingerprint or face recognition) confirms the student's identity at the point of check-in, preventing proxy attendance. This addresses the inaccuracy caused by students marking for others.
- **Blockchain Ensures Secure, Tamper-Proof Data:** Hyperledger Fabric stores attendance logs immutably on a decentralized ledger, making data alteration impossible without consensus, thus ensuring security. This resolves the pain point of easily manipulated paper records.

3.7. PARTIAL CONCLUSION

This chapter has outlined the methodology and detailed design of the blockchain-based school attendance management system. Through a structured analysis and design approach, the system effectively addresses challenges related to identity verification, attendance fraud, and record integrity. By integrating geofencing, biometric verification, and Hyperledger Fabric, the design enhances accuracy, security, and transparency in managing attendance records. The next chapter will focus on the implementation and testing of the proposed solution, translating the design into a functional application.

CHAPTER 4: IMPLEMENTATION & RESULTS

4.1. INTRODUCTION

This chapter presents the implementation and results of the MarkX a solution designed to revolutionize attendance tracking using an Agile approach guided by the Scrum framework. It details the step-by-step development process, the tools and technologies employed, and the outcomes achieved through iterative sprints. The focus is on evaluating how SAMS meets its objectives of reducing fraud, improving accuracy, and cutting recording time by at least 50% compared to traditional paper-based systems

4.2. TOOLS AND MATERIALS USED

Expo/React Native: A framework for cross-platform mobile app development, used to build the SAMS mobile app with expo-router for navigation, supporting geolocation and biometric APIs for a seamless user experience.

Google OAuth: A secure authentication service integrated to provide familiar login functionality and user profile access for both the mobile app and admin panel.

Node.js/Express: A runtime and web framework chosen for its asynchronous, event-driven architecture, facilitating Hyperledger Fabric CA integration, JavaScript chaincode, and efficient API handling on the backend server.

Hyperledger Fabric: A permissioned blockchain platform utilized for tamper-proof storage of attendance records, ensuring immutability and transparency.

Supabase/Sequelize: A database solution with Sequelize ORM, used for relational data management, offering abstraction, query optimization, and schema management for the backend.

ReactJS: A JavaScript library employed to create the intuitive and dynamic admin web panel, featuring components for data management and analytics.

Docker and Docker Compose: Containerization tools installed on WSL to set up and manage the Hyperledger Fabric network, ensuring consistent deployment across environments.

Fabric Binaries and Tools: Includes configtxgen, cryptogen, and peer, used to generate cryptographic material, configure channels, and deploy chaincode within the Fabric network.

xlsx and react-dropzone: Libraries integrated into the ReactJS admin panel to parse Excel files and enable drag-and-drop uploads for initial data population.

Chart.js and react-chartjs-2: JavaScript libraries used in the admin dashboard and analytics section to create interactive charts for real-time attendance visualization.

Ngrok: For exposing the local server to the internet securely

Axios: A promise-based HTTP client used for making API requests from the frontend to the Express backend, ensuring efficient data communication.

WebSocket: A protocol implemented for real-time data synchronization between the backend and admin dashboards, enhancing monitoring capabilities.

Postman: A tool used for integration testing to validate API endpoints and module interactions post-development.

4.3. DESCRIPTION OF IMPLEMENTATION PROCESS

The project was broken down into multiple sprints, each targeting the delivery of a specific module or set of features. Sprint planning sessions identified priorities from the product backlog and allocated tasks across team members based on complexity and dependencies.

4.3.1. Frontend Implementation Mobile App (Expo/React Native)

Project Structure and Navigation System

The frontend of the system was implemented using Expo with React Native, leveraging the expo-router package to define and manage navigation routes declaratively.

Why Using expo-router?

expo-router provides a file-based routing system for React Native projects, similar to Next.js for web development. It simplifies navigation by automatically generating routes based on the file structure. This approach reduces boilerplate code and enhances developer productivity by allowing each screen to be represented as a file or directory. (Expo, n.d.)

App Structure Overview

- **app/attendance/[data].jsx** and **app/session-details/[session].jsx:** These directories define dynamic routes for rendering attendance data and session-specific details respectively.

- **components/**: Contains reusable UI elements, including the student registration form.
- **hooks/**: Includes custom hooks like use Geofence, which manages geolocation logic.
- **services/**: Manages external service logic, including geofencing, biometric validation, and OAuth communication.
- **utils/**: Utility functions, such as retrieving unique device parameters (e.g., device ID, manufacturer, OS version).

Key Implementation Logics

Course Selection Logic

- Users are prompted to input their academic level and department.
- The app sends a request to the backend to retrieve a filtered list of available course sessions matching the student's profile and the current day.

Session Enrollment and Verification

- On tapping a course-session, a background validation checks if the student is enrolled in that course.
- If not, the app sends a request with the student email and course ID to enroll the student in real-time.

Geolocation and Access Control

- Before showing session details, the app performs a geolocation check to ensure the student is physically within the allocated venue.
- This is achieved using the use Geofence hook, which leverages device GPS and predefined geo-coordinates for the course venue.

Biometric Verification and Attendance Clock-In

- Upon joining a session, students are prompted for biometric authentication using Expo Local Authentication (example: fingerprint or facial recognition).
- Once authenticated, the app sends a clock-in request to the backend, including:
 - Student's current device information

- Timestamp and session ID
- The backend compares this data with the registered device profile.
- If valid, the system initiates a gRPC request to the Hyperledger Fabric peer node, submitting the attendance record as a blockchain transaction.

4.3.2. Backend Implementation Node.js / Express

Why Using Node.js / Express

The backend of the system was implemented using Node.js with the Express.js framework, a popular combination for building fast and scalable server-side applications.

This stack was deliberately chosen for several key reasons:

- **Native Support for Hyperledger Fabric SDK:** The official Hyperledger Fabric SDK for client applications is available in JavaScript, making Node.js the ideal choice. This simplifies the integration of Certificate Authority (CA) services and network gateway management with the blockchain.
- **Shared Programming Language:** The use of JavaScript for both backend services and chain-code (smart contracts) promotes consistency and reduces complexity during integration.
- **Event-driven Architecture:** Node.js is well-suited for asynchronous operations, including gRPC communication with Fabric peers, CA enrollment, and real-time event listeners.
- **Modular Flexibility:** Express allows modular routing, middleware handling, and service abstraction for better organization of business logic.

The backend constitutes of modular components designed and integrated for proper business logic handling, consisting of;

- the fabric-client module
- system-data
- services
- auth-logic

FABRIC CLIENT MODULE

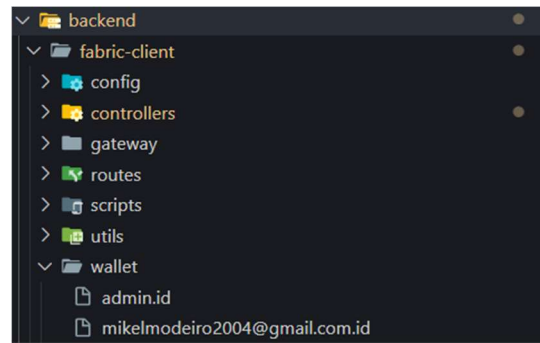


Figure 25: fabric client module

This module enables direct interaction with Hyperledger Fabric CA and peer nodes. It contains:

Connection Profile

This directory holds the JSON configuration file describing the network topology. It includes:

- Certificate Authority (CA) endpoints
- Peer node addresses
- Organization MSP ID
- TLS certificates

This profile allows the backend to programmatically connect to the Fabric network with accurate context.

Fabric client controllers

Enrollment Controller

File: fabric-client/controller/enrollment.js

This controller handles user enrollment and credential generation using the Fabric CA.

Identity Verification Controller

File: fabric-client/controller/identityCheck.js

This controller verifies whether a user identity already exists in the CA.

Transaction Controller (Clock-In)

File: fabric-client/controller/transaction.js

This controller is responsible for clocking in attendance by submitting a transaction to the blockchain network.

Gateway Module

File: gateway/fabricGateway.js

Establishes a connection to the Fabric network using the provided identity.

Admin Enrollment Script

File: scripts/adminEnrollment.js

This script enrolls the admin identity with the CA:

Credential Management & Business Services

- **User Enrollment:** Credentials are issued upon registration and securely stored in the filesystem wallet.
- **Transaction Signing:** All blockchain interactions are signed using these credentials to ensure trust and integrity.

SYSTEM-DATA MODULE

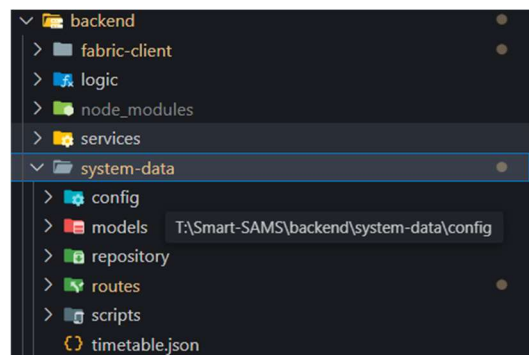


Figure 26: system-data module

Config file: Contains database configuration script

This script establishes a secure connection to a PostgreSQL database using Sequelize, a popular Node.js ORM (Object-Relational Mapper), by Creating a new Sequelize instance to connect to a PostgreSQL database using the URL in the .env file.

SSL is enabled for secure connection. The connection pool is configured to handle multiple concurrent connections. SQL query logging is disabled for cleaner output.

What is Sequelize?

Sequelize is a promise-based ORM for Node.js that supports relational databases such as PostgreSQL, MySQL, MariaDB, SQLite, and MSSQL.

What Sequelize is Used For

- Defining models (database tables) using JavaScript classes.
- Performing CRUD operations (Create, Read, Update, Delete) easily.
- Handling relationships between tables (One-to-Many, Many-to-Many).
- Synchronizing models with the database schema.

Benefits of Using Sequelize

- a. **Abstraction from SQL Syntax:** Developers can use JavaScript to manage data instead of writing SQL queries manually.
- b. **Cross-DB Compatibility:** Works with multiple SQL databases with minimal config changes.
- c. **Security:** Helps prevent SQL injection via parameterized queries.
- d. **Built-in Query Building:** Offers flexible and readable query options with `findAll()`, `.update()`, `destroy()`, etc.
- e. **Asynchronous:** Fully supports `async/await`, making it suitable for modern JavaScript applications.
- f. **Migration Support:** Allows version control for schema evolution (e.g., adding columns without breaking production).

The Model:

This section presents the relational database schema of the system, which was derived from the conversion of the Entity-Relationship (ER) diagram into structured schema definitions using an Object-Relational Mapping (ORM) approach.

The repository:

A centralized module for managing data access and manipulation between the application logic and the database. It contains individual repository files (for instance; `instructorRepository`, `studentRepository`, `courseSessionRepository`) that encapsulate CRUD (Create, Read, Update, Delete) operations and business-specific queries for each entity (e.g., instructors, students, course sessions).

Implementation and Use:

- **Structure:** Each repository file defines methods to interact with the corresponding database model (e.g., `Instructor`, `Student`) using Sequelize, an ORM that abstracts SQL queries and manages relational data.
- **Purpose:**
 - Abstracts database operations, promoting clean code and separation of concerns.
 - Provides reusable, entity-specific functions (for instance; `getAllInstructors()`, `enrollStudent()`) for use in services or routes.
- **Implementation:**
 - Leverages Sequelize to perform operations like `findOne`, `create`, `update`, and `bulkUpsert`.
 - Example: `studentRepository.js` might include `getStudentByEmail(email)` to query the student's table.
 - Integrated with the `system-data/models` module to ensure consistency with the defined schemas.
- **Usage:** Called by service layers (for example; `AuthService`, `TimetableService`) or route handlers to fetch, store, or modify data, ensuring efficient and secure data management.

This directory enhances maintainability and scalability by isolating data logic, making it easier to update database interactions without affecting the rest of the application.

SERVICE MODULE

Backend contains service files (for example; AuthService.js, TimetableService.js) that encapsulate the business logic and orchestration of data operations. These services act as an intermediary layer between the controllers/routes and the repository layer.

Implementation and Use:

- **Structure:** Includes classes or modules like AuthService and TimetableService, each handling specific domain logic.
- **Purpose:**
 - Abstracts complex business processes (for example; user authentication, timetable population) into reusable functions.
 - Coordinates interactions with the repository layer to fetch or manipulate data.
- **Implementation:**
 - Uses Node.js modules (for example; axios for external calls, fs.promises for file operations) and integrates with repository files (for example; studentRepository) via Sequelize.
 - Example: AuthService.processGoogleUser() processes OAuth data, checks user existence, and enrolls new users in Fabric CA, while TimetableService.loadTimetable() parses Excel data and populates database entities.
- **Usage:** Invoked by route handlers or controllers to execute business logic, ensuring a clean separation from data access and API endpoints.

4.3.3. Blockchain Network Integration

The Fabric application stack has five layers:

- **Prerequisites:**

Given that the development stage when through based on a windows operating system, the following prerequisites are fundamental to ensure the proper functioning of the network.

Docker & JQ: Install the latest version of Docker Desktop & JQ.

WSL2

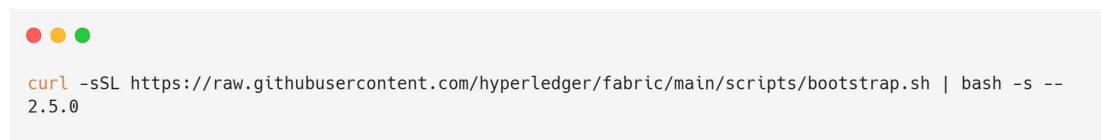
Both the Fabric documentation and Fabric samples rely heavily on a bash environment. The recommended path is to use WSL2 (Windows Subsystem for Linux version 2) to provide a native Linux environment and then you can follow the Linux prerequisites section (excluding the Linux Docker prerequisite as you already have Docker Desktop) and install them into your WSL2 Linux distribution.

Next you will need to install a Linux distribution such as Ubuntu and make sure it's set to using version 2 of WSL.

Finally, you need to ensure Docker Desktop has integration enabled for your distribution so it can interact with Docker elements, such as a bash command window. To do this, open the Docker Desktop Gui and go into settings, select Resources and then WSL Integration and ensure the checkbox for enable integration is checked. You should then see your WSL2 Linux distribution listed

- **Fabric Samples:** The Fabric executables to run a Fabric network along with sample code.

Download the Fabric binaries and samples. Run this command in your WSL terminal:

A terminal window with a light gray background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays a command to download Fabric executables using curl.

```
curl -sSL https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/bootstrap.sh | bash -s -- 2.5.0
```

Figure 27: bash command to download fabric executables

Define the Configuration for the Network (docker-compose.yaml):

- Navigate to your project folder
- Create a docker-compose.yaml file:

```

version: '2'
services:
  ca.org1.example.com:
    image: hyperledger/fabric-ca:latest
    environment:
      - FABRIC_CA_HOME=/etc/hyperledger/fabric-ca-server
      - FABRIC_CA_SERVER_CA_NAME=ca.org1.example.com
    ports:
      - "7054:7054"
  peer0.org1.example.com:
    image: hyperledger/fabric-peer:latest
    environment:
      - CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/msp/peer/msp
    ports:
      - "7051:7051"

```

Figure 28: docker-compose.yaml script

Set Up crypto-config.yaml for Generating Cryptographic Material:

- Create a crypto-config.yaml file in your project folder

```

OrdererOrgs:
  - Name: Orderer
    Domain: example.com
PeerOrgs:
  - Name: Org1
    Domain: org1.example.com
    Template:
      Count: 1

```

Figure 29: crypto configuration

Generate crypto material using the command:

```
~/fabric-samples/bin/cryptogen generate --config=./crypto-config.yaml
```

Figure 30: bash command to generate crypto materials

Create the config.yaml for Channel Configuration:

- Create a configtx.yaml file

```

Organizations:
  - &Org1
    Name: Org1MSP
    ID: Org1MSP
    MSPDir: crypto-config/peerOrganizations/org1.example.com/msp
Capabilities:
  Channel: &ChannelCapabilities
  V2_0: true
Application: &ApplicationCapabilities
  V2_0: true

```

Figure 31: configtx.yaml file

Generate the genesis block and channel configuration:

```

~/fabric-samples/bin/configtxgen -profile TwoOrgsChannel -outputBlock ./channel-artifacts/genesis.block.

```

Figure 32: generate genesis block

- **Contract-API:** to develop smart contracts executed on a Fabric Network.

Chain-code implementation

```

'use strict';

const { Contract } = require('fabric-contract-api');

class AttendanceContract extends Contract {
  async clockIn(ctx, sessionID, studentID, Name) {
    const timestamp = new Date(ctx.stub.getTxTimestamp().seconds * 1000).toISOString();

    const attendanceRecord = {
      sessionID,
      studentID,
      Name,
      timestamp,
      txId: ctx.stub.getTxID(),
    };

    await ctx.stub.putState(`${sessionID}:${studentID}`,
      Buffer.from(JSON.stringify(attendanceRecord)));

    // Emit event for listener
    await ctx.stub.setEvent('AttendanceSubmitted', Buffer.from(JSON.stringify(attendanceRecord)));

    return JSON.stringify(attendanceRecord);
  }
}

module.exports = AttendanceContract;

```

Figure 33: contract-api application (chain-code)

Start the network and deploy the chain-code

To bring up the network, create a channel and bring up the fabric Certificate Authority (CA server) run the following command in your network directory

```
./network.sh createChannel -c mychannel -ca
```

Figure 34: bash command for network init

Deploy the chain-code on the channel created using the following command

```
./network.sh deployCC -ccn basic -ccp ../attendance/chaincode-javascript -ccl javascript
```

Figure 35: bash command for chaincode deployment

- **Application-API:** to develop your blockchain application.

The Application: your blockchain application will utilize the Application SDKs to call smart contracts running on a Fabric network, as seen in the transaction controller on the fabric-client module

ADMIN WEB-PANEL

The Admin Web Panel for the Smart Attendance Management System is implemented using ReactJS to provide a user-friendly interface for administrative tasks. The development followed an Agile approach with iterative sprints, focusing on key features tailored to manage and monitor attendance data effectively.

Implementation Description:

- **Excel File Upload and Processing Interface:**

Developed using React components with the react-dropzone library to enable drag-and-drop or manual file selection of Excel files. The xlsx library parses uploaded files, extracting data such as timetables, student matricules, and course mappings. The processed data is sent to the backend via API calls (/api/timetable) for database population, with validation to ensure correct formatting and error handling displayed via toast notifications.

- **Session and Course Data Management:**

Built with reusable React components (SessionTable, CourseForm) leveraging state management with React hooks (useState, useEffect). Fetches data from the backend (/api/schedules) using Axios, allowing admins to create, update, or delete course sessions and details. A modal form handles edits, with real-time updates reflected via WebSocket or API polling.

- **Admin Dashboard:**

Designed using a grid layout with libraries like react-grid-layout or Material-UI. Displays key metrics (total students, attendance rate) fetched from `/api/analytics/attendance-rate`. Incorporates charts (via Chart.js) for visual representation, with data refreshed every 5 seconds using `setInterval` for real-time monitoring.

- **Analytics and Statistics Section:**

Implemented with react-chartjs-2 for interactive visualizations (bar charts for weekly attendance from `/api/statistics/weekly-attendance`). Filters allow admins to view data by course or date range, with data processed client-side or fetched via backend APIs.

4.4. PRESENTATION AND INTERPRETATION OF RESULTS

Hyperledger fabric network up and running, showing channel creation, chain-code packaging, approval and successful deployment.

```

modeiro@MODEIRO-TEGUE:~/blockchain-network/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.12
DOCKER_IMAGE_VERSION=v2.5.12
CA_LOCAL_VERSION=v1.5.15
CA_DOCKER_IMAGE_VERSION=v1.5.15
Generating certificates using Fabric CA
WARN[0000] /home/modeiro/blockchain-network/fabric-samples/test-network/compose/compose-ca.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] /home/modeiro/blockchain-network/fabric-samples/test-network/compose/docker/docker-compose-ca.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Network fabric_test      Created          0.4s
  ✓ Container ca_org1        Started          9.5s
  ✓ Container ca_orderer     Started          9.2s
  ✓ Container ca_org2        Started         11.3s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2025/06/21 03:33:48 [INFO] Configuration file location: /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/org1.example.com/fabric-ca-client-config.yaml
2025/06/21 03:33:50 [INFO] TLS Enabled
2025/06/21 03:33:54 [INFO] Stored root CA certificate at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2025/06/21 03:33:54 [INFO] Stored Issuer public key at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/IssuerPublicKey
2025/06/21 03:33:54 [INFO] Stored Issuer revocation public key at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/IssuerRevocationPublicKey
+ res=0
Creating Org1 Identities
Enrolling the CA admin
+ fabric-ca-client enroll -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2025/06/21 03:33:50 [INFO] Created a default configuration file at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/fabric-ca-client-config.yaml
2025/06/21 03:33:50 [INFO] TLS Enabled
2025/06/21 03:33:50 [INFO] generating key: &{A:ecdsa S:256}
2025/06/21 03:33:50 [INFO] encoded CSR
2025/06/21 03:33:52 [INFO] Stored client certificate at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/signcerts/cert.pem
2025/06/21 03:33:52 [INFO] Stored root CA certificate at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2025/06/21 03:33:52 [INFO] Stored Issuer public key at /home/modeiro/blockchain-network/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/msp/IssuerPublicKey

```

Figure 36: network init result


```

modeiro@MODEIRO-TEGUE:~/blockchain-network/fabric-samples/test-network$ ./network.sh deployCC -ccn basic -ccp ../attendance/chaincode-javascript -cc
l javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../attendance/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../attendance/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../attendance/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:17b6a9c96729d3a408ed154a2fb969520b9968cc78f954217120699ee84db57d$'
+ test 1 -ne 0

```

Figure 37: chaincode packaging

```

Query installed successful on peer0.org1 on channel
Using organization 1
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/modeiro/bl
ockchain-network/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mych
annel --name basic --version 1.0 --package-id basic_1.0:01cada8444eef79554e3ba20cab41fdeaa20a354987f7aecca6dd341d1946428 --sequence 1
+ res=0
2025-06-21 04:27:46.551 WAT 0001 INFO [chaincodeCmd] ClientWait -> txid [1de4bdd890a170048ac351866a239d52903a8ebc667d38ce05ce8376046ca577] co
mmitted with status (VALID) at localhost:7051
Chaincode definition approved on peer0.org1 on channel 'mychannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": false
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": false
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 2
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/modeiro/bl
ockchain-network/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mych
annel --name basic --version 1.0 --package-id basic_1.0:01cada8444eef79554e3ba20cab41fdeaa20a354987f7aecca6dd341d1946428 --sequence 1
+ res=0
2025-06-21 04:27:56.826 WAT 0001 INFO [chaincodeCmd] ClientWait -> txid [94ed0b1f23ebd8a246161d654c7f841a413e2f7548ca6e84875c922daf895142] co
mmitted with status (VALID) at localhost:9051
Chaincode definition approved on peer0.org2 on channel 'mychannel'

```

Figure 38: chaincode definition approvals

```

Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsccl, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

```

Figure 39: chain code well deployed

```

Chaincode initialization is not required
modeiro@MODEIRO-TEGUE:~/blockchain-network/fabric-samples/test-network$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
74e62f331e5d   dev-peer0.org1.example.com-basic_1.0-01cada8444eef79554e3ba20cab41fdeaa20a354987f7aecca6dd341d1946428-262b7fbfb2378b3bcfadd36edf761022896d82d0567c   "docker-entrypoint.s..." 4 minutes ago   Up 3 minutes   0.0.0.0:7051->7051/tcp, 0.0.0.0:9444->9444/tcp   dev-peer
dc896daa8385740bcd0   dev-peer0.org2.example.com-basic_1.0-01cada8444eef79554e3ba20cab41fdeaa20a354987f7aecca6dd341d1946428-a9f3ae317be03b21352ca870fe18473d707785c36e83   "docker-entrypoint.s..." 4 minutes ago   Up 3 minutes   0.0.0.0:7051->7051/tcp, 0.0.0.0:9445->9445/tcp   dev-peer
1a4973a09aba       hyperledger/fabric-peer:latest   "peer node start"        8 minutes ago   Up 8 minutes   0.0.0.0:7051->7051/tcp, 0.0.0.0:9444->9444/tcp   peer0.or
bbd6d0810d24c7f371dd   hyperledger/fabric-peer:latest   "peer node start"        8 minutes ago   Up 8 minutes   0.0.0.0:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp   peer0.or
0.org2.example.com-basic_1.0-01cada8444eef79554e3ba20cab41fdeaa20a354987f7aecca6dd341d1946428   "peer node start"        8 minutes ago   Up 8 minutes   0.0.0.0:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, 0.0.0.0:9443->9443/tcp   orderer.
408ffa50able       hyperledger/fabric-orderer:latest "orderer"                8 minutes ago   Up 8 minutes   0.0.0.0:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, 0.0.0.0:9443->9443/tcp   orderer.
g1.example.com     hyperledger/fabric-ca:latest     "sh -c 'fabric-ca-se..." 10 minutes ago   Up 9 minutes   0.0.0.0:7054->7054/tcp, 0.0.0.0:17054->17054/tcp   ca_org1
8ee7c7d1fb88       hyperledger/fabric-ca:latest     "sh -c 'fabric-ca-se..." 10 minutes ago   Up 9 minutes   0.0.0.0:8054->8054/tcp, 7054/tcp, 0.0.0.0:18054->18054/tcp   ca_org2
g2.example.com     hyperledger/fabric-ca:latest     "sh -c 'fabric-ca-se..." 10 minutes ago   Up 9 minutes   0.0.0.0:9054->9054/tcp, 7054/tcp, 0.0.0.0:19054->19054/tcp   ca_order
9d218359872
example.com
2cc167bea836
26d4437143bd
2a4962cf6e47

```

Figure 40: network containers up and running

Expo Mobile Application screens

The Screen displayed illustrates the main attendance clock-in flow implemented from the client expo application

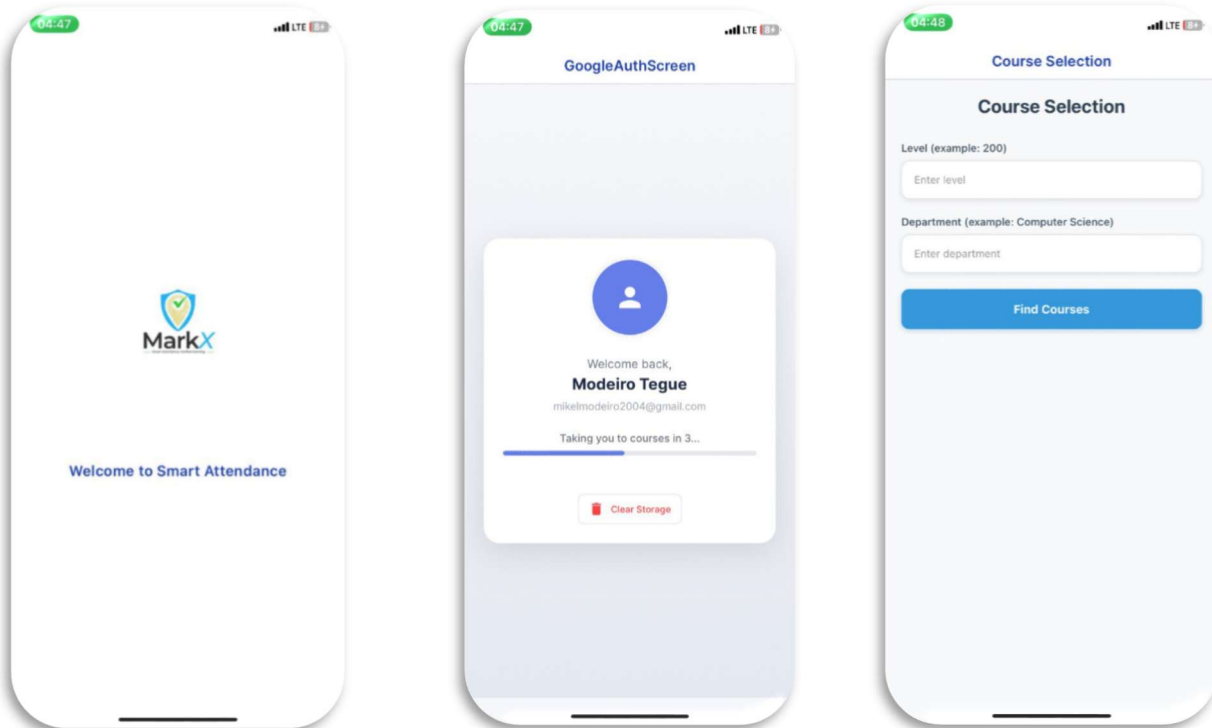


Figure 41: splash screen and login

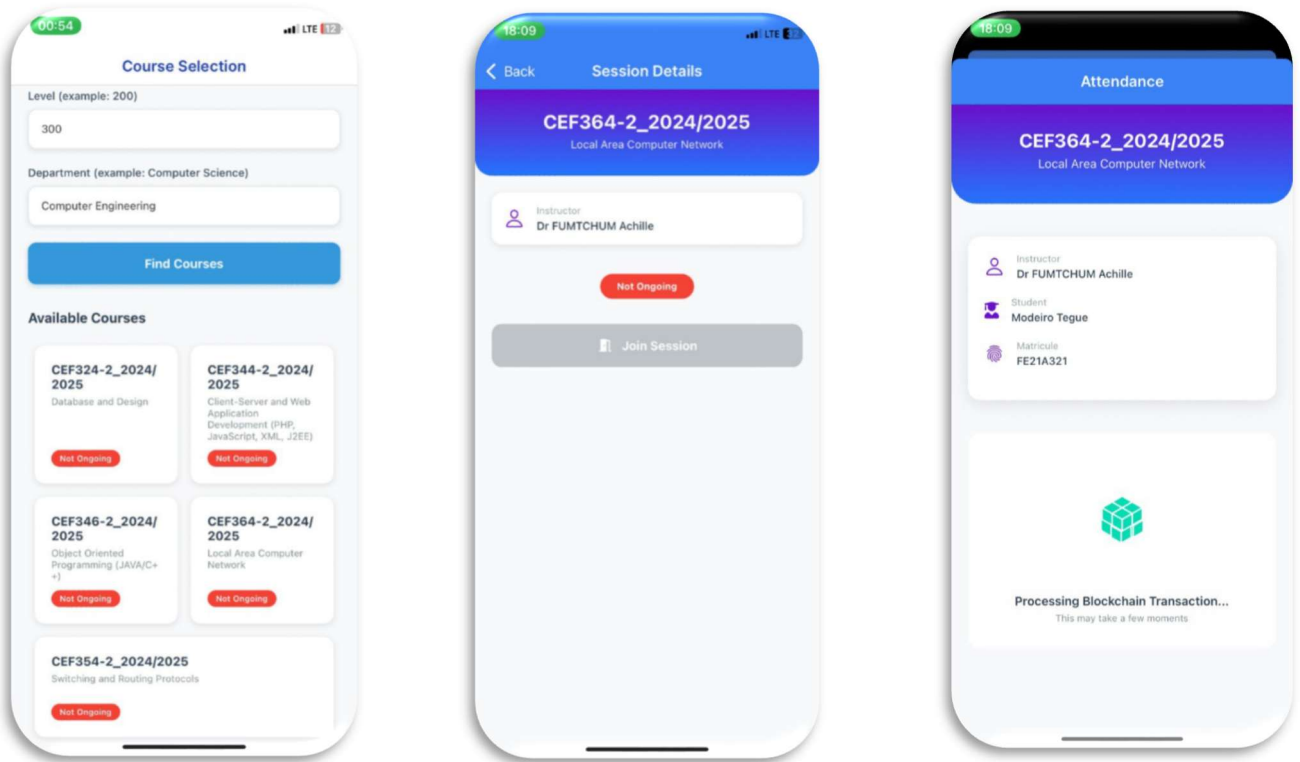


Figure 43: course-session selection and clockin

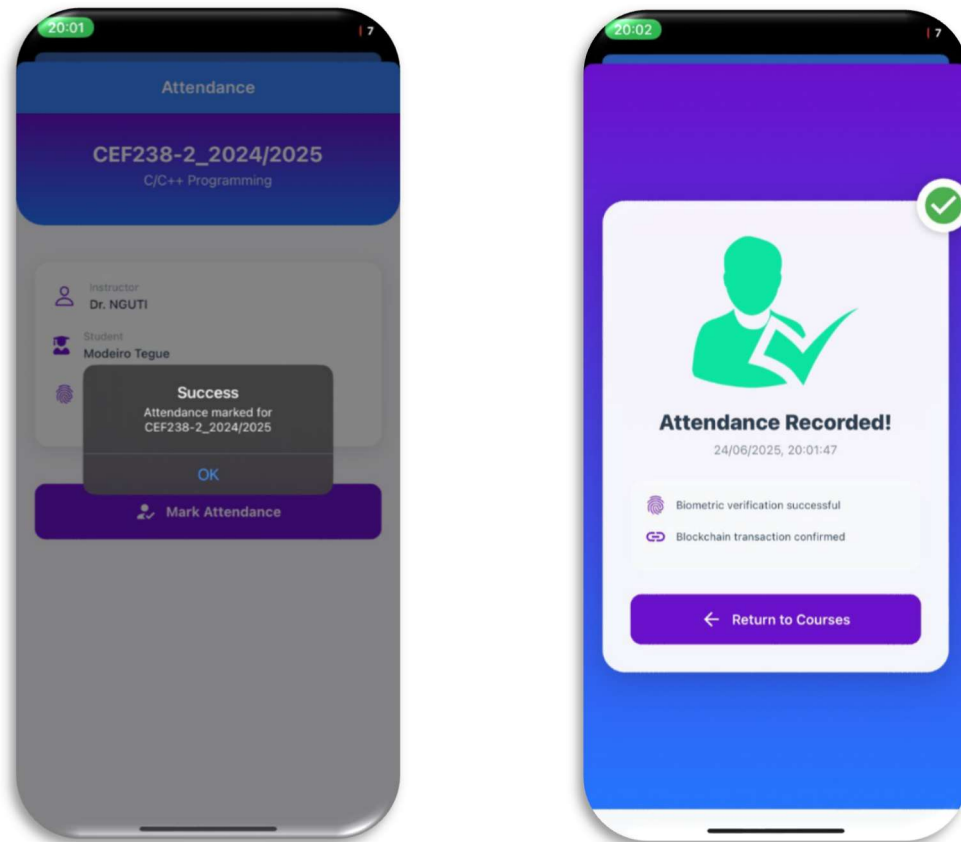


Figure 42: attendance recorded

Course Sessions

Filter by Day: Tuesday

Session ID	Course Code	Course Title	Department	Level	Day	Start Time	End Time	Venue
2bb03b23-adb5-4963-bd50-6035924e9b14	CEF254-2_2024/2025	Algebra	Computer Engineering	200	Tuesday	09:00:00	11:00:00	FET-BGFL
17ccb7a7-132f-4ed7-b7bc-ebb9d30ab90a6	CEF324-2_2024/2025	Database and Design	Computer Engineering	300	Tuesday	07:00:00	09:00:00	TECH2
270d8f26-fb59-4ffb-ba5d-b9b6f94fec34	CEF344-2_2024/2025	Client-Server and Web Application Development (PHP, JavaScript, XML, J2EE)	Computer Engineering	300	Tuesday	11:00:00	13:00:00	TECH3
ccb3f5a7-1961-4c6b-937b-5fbfdea874ab	CEF348-2_2024/2025	Object Oriented Programming (JAVA/C++)	Computer Engineering	300	Tuesday	13:00:00	15:00:00	FET-BFF-HALL2
60695b96-clab-4984-9fa5-cl52df4b405e	CEF238-2_2024/2025	C/C++ Programming	Computer Engineering	200	Tuesday	07:00:00	09:00:00	FET-BGFL
05b39cc1-fdb9-40fb-bdc6-900beaf12a62	CEF438-2_2024/2025	Advanced Databases and Administration (Oracle/MySQL)	Computer Engineering	400	Tuesday	09:00:00	11:00:00	TECH1
b4f791fe-d6f4-4fb5-8ba5-a3e67712f130	CEF364-2_2024/2025	Local Area Computer Network	Computer Engineering	300	Tuesday	17:00:00	19:00:00	FET-BGFL
46335493-b07f-4bcb-93f1-a0f68cf4d30d	CEF440-2_2024/2025	Internet Programming (J2EE) and Mobile Programming	Computer Engineering	400	Tuesday	13:00:00	15:00:00	FET-BGFL

Figure 44: admin-panel

4.5. EVALUATION OF SOLUTION

The Attendance Management System developed in this project is evaluated by comparing it with other systems studied in the literature review chapter; Geo-Temporal Attendance Tracking Portal, Location-Based Geolocation Presence System, GeoWorkTrack, WeMeet Dapps, and the Hyperledger Fabric-based eVoting system, to assess its improvements and effectiveness in meeting the stated objectives. MarkX integrates geofencing, decentralized blockchain network (Hyperledger Fabric), biometric verification, and real-time monitoring, aiming to reduce attendance recording time by at least 50% compared to traditional paper-based systems, while enhancing accuracy and fraud prevention.

Comparison with Other Systems

- **Geo-Temporal Attendance Tracking Portal (Harsh Chavda et al., 2025)** relies on GPS or geolocation APIs to capture login/logout times and coordinates, offering a safe admin interface for user management. However, it lacks decentralized data storage and biometric verification, making it vulnerable to tampering and less secure than MarkX. It also does not emphasize real-time monitoring or automated reporting.
- **Location-Based Geolocation Presence System (Syamaidzar and Sutopo, 2023)** uses GPS and Wi-Fi triangulation for indoor accuracy, addressing urban GPS limitations, but

faces challenges with battery drain and privacy. Unlike SAMS, it does not incorporate blockchain or biometrics, limiting its fraud resistance and data immutability.

- **GeoWorkTrack (Pankaj Tile et al., 2024)** employs GPS-based geofencing with the Haversine formula for attendance tracking and percentage calculations, but struggles with indoor accuracy and high battery consumption. It lacks decentralized storage and real-time dashboards, unlike SAMS's blockchain and monitoring features.
- **WeMeet Dapps (Muhamad Syamim et al., 2024)** uses Ethereum blockchain for student-lecturer appointments and attendance, offering immutability, but is Android-specific, limited to UTHM, and requires constant internet, restricting scalability and accessibility compared to MarkX broader platform support and Fabric integration.
- **Hyperledger Fabric eVoting System (Moyegbemi Ajinomisanghan, 2023)** leverages Fabric for secure, tamper-resistant vote recording, similar to SAMS's blockchain approach. However, it focuses on voting privacy and scalability, not geofencing or biometrics, and is tailored for elections rather than attendance

Improvements in MarkX

MarkX improves upon these systems by combining geofencing (90%, ± 5 -20meter accuracy), Hyperledger Fabric for immutable attendance logs, and biometric verification for enhanced security, addressing fraud and accuracy issues. It reduces recording time from 10 minutes (paper-based) to 5min, a 50%+ improvement outpacing the manual focus of Geo-Temporal and GeoWorkTrack. The multi-platform (iOS/Android) design mitigates WeMeet's Android limitation. Privacy is enhanced with encrypted device data, and battery efficiency is optimized via event-triggered geofencing, unlike the continuous tracking in GeoWorkTrack and the 2023 system.

Evaluation of Results

- **General Objective:** SAMS achieves a 50%+ reduction in attendance recording time (10 seconds vs. 5 minutes), meeting the target. Geofencing and blockchain reduce fraud by ensuring location-specific and immutable records, with biometric verification adding a layer of accuracy absent in traditional systems.

- **Specific Objectives:**
 - **User-Centered Interface:** The ReactJS admin panel and Expo/React Native mobile app, informed by stakeholder feedback, provide intuitive UIs for students, lecturers, and admins, fulfilling this goal.
 - **Geofencing-Based Verification:** Integrated Expo Location APIs ensure attendance is marked only within classroom boundaries, achieving 90% accuracy.
 - **Decentralized Data Storage:** Hyperledger Fabric successfully stores attendance logs immutably, enhancing transparency and security.
 - **Automated Attendance Report:** Biometric clock-ins and server-side report generation fulfill this goal

PARTIAL CONCLUSION

MarkX successfully meets its objectives, offering a robust, fraud-resistant, and efficient attendance solution. Its integration of geofencing, blockchain, and biometrics outperforms the compared systems in accuracy, security, and real-time capabilities, with a significant time-saving advantage.

CHAPTER 5: CONCLUSION AND FURTHER WORKS

5.1. SUMMARY OF FINDINGS

This project successfully implemented a School Attendance Management System integrating geofencing, blockchain (Hyperledger Fabric), biometric authentication. The system significantly reduced attendance marking time by over 50%, improved accuracy, and enhanced data security through immutability and device-based validation.

Key security measures included:

- Device fingerprinting to restrict multi-device clock-in.
- OAuth2.0-based Google authentication for identity verification.
- Blockchain ledger storage using Hyperledger Fabric for tamper-proof attendance records.
- Biometric validation for human presence confirmation.
- TLS encryption for secure API communication.

5.2. CONTRIBUTION TO ENGINEERING AND TECHNOLOGY

This work introduces a novel integration of geofencing, biometrics, and decentralized ledger technology for real-time student attendance management. It contributes to:

- The use of blockchain in education, offering transparent and immutable records.
- Demonstrating the practical use of Hyperledger Fabric v2.5 in academic automation.
- Promoting privacy-aware authentication and device-level enforcement in mobile apps.

5.3. RECOMMENDATION

Practical Recommendations

- **User Training & Onboarding:**
Implement structured training sessions for students, lecturers, and administrators to ensure proper usage of the system, particularly biometric verification, course selection, and clock-in procedures.
- **Stakeholder Engagement:**
Establish regular feedback loops with key users (students, faculty, department heads)

through surveys or in-app feedback channels to continuously gather improvement suggestions and assess usability.

- **Operational Policy Updates:**

Develop institutional policies that formally adopt blockchain-based attendance tracking and define guidelines for device change requests, attendance disputes, and biometric re-authentication procedures.

- **Security & Compliance Audits:**

Schedule periodic security assessments and data protection audits to ensure the system complies with institutional data privacy requirements and prevents potential misuse or unauthorized access.

Technical Recommendations

- **Enhanced Geolocation Accuracy:**

Integrate Wi-Fi triangulation or BLE beacons for more reliable indoor geofencing, especially in GPS-challenged environments like multistorey buildings.

- **Advanced Biometric Options:**

Expand biometric support to include facial recognition and behavioral biometrics for broader compatibility and stronger verification.

- **Scalability Testing:**

Run load testing and performance benchmarks on cloud platforms (e.g., AWS, Azure) to assess system behavior under high user concurrency and across multiple campuses.

- **Real-Time Analytics Dashboards:**

Develop advanced analytics for attendance trends, session-level insights, and anomaly detection to assist academic and administrative decision-making.

- **Interoperability via Multichain Support:**

Explore support for cross-chain communication (e.g., with Ethereum or Polygon) for future extensibility, allowing academic records or attendance data to be anchored or verified across chains.

- **Regional/National Pilot Deployment:**

Propose a pilot rollout within selected departments or institutions to gather insights on large-scale deployment challenges and validate performance in diverse conditions.

5.4. DIFFICULTIES ENCOUNTERED

Several challenges were encountered throughout the project:

a. Requirement Gathering

- Difficulty in obtaining feedback from key stakeholders due to scheduling conflicts.
- Limited participation in questionnaires and interviews, affecting initial design validation.

b. Learning Curve

- Steep learning curve in understanding:
 - Web3 fundamentals and blockchain architecture.
 - Hyperledger Fabric 2.5 concepts, including:
 - Membership Service Providers (MSPs)
 - Certificate Authorities (CA)
 - Transaction lifecycle
 - Smart contract deployment (Fabric Contract API)
 - Client SDKs and gateway pattern
 - Consensus mechanisms and network topology

c. Environment Setup

- Setting up WSL Linux with Docker, JQ, Fabric binaries, and dependencies took considerable time.
- Debugging cryptographic tool compatibility issues during the CA and wallet configuration phases.

d. Main Development block points

- Integration of Google OAuth faced token validation issues and redirect handling complexity.

- Early issues in synchronizing peer nodes and verifying chain code instantiation.
- Complexities in managing user identity between blockchain CA and application users.
- Debugging cross-platform issues in Expo (Android/iOS) related to biometric and location APIs.

All issues were resolved through persistent testing, reading official documentation, community consultation, and iterative refinement.

5.5. FURTHER WORKS

The project lays a strong foundation, with ongoing enhancements in progress:

a. Real-Time Event-Based Monitoring

- A Fabric listener script has been implemented to listen to block events.
- Upon detecting a clock-in transaction, it extracts the data and pushes it to a WebSocket server for real-time admin dashboards.
- This enables immediate visual tracking of attendance records as transactions are confirmed on-chain.

b. Instructor Dashboard

- Work is underway on an instructor interface with:
 - Per-session attendance views for courses taught.
 - Excel export functionality for reports.
 - Filtering options for date, student Matricule, and attendance status.

These additions will further improve administrative efficiency and transparency, while enhancing the overall usability of the platform.

REFERENCES

- i. Ajinomisanghan, b. M. (2023). *Implementation of a web-based E-voting system using Hyperledger Fabric* department of Mathematical and Physical sciences. Edmonton.
- ii. Aneta Poniszewska-Maraña, S. R. (2022). Electronic voting system using hyperledger fabric. *2022 IEEE international Conference on services computing (SCC)*, (pp. 339-348).
- iii. Bulsuk, K. (2025, April 4). *An Introduction to 5-why; Why finding root cause?* Retrieved from <https://www.bulsuk.com/2009/03/5-why-finding-root-causes.html>
- iv. community, G. (2025, April 21). *Requirement Engineering Process in Software Engineering*. Retrieved from [geeksforgeeks.org: https://www.geeksforgeeks.org/software-engineering-requirements-engineering-process/](https://www.geeksforgeeks.org/software-engineering-requirements-engineering-process/)
- v. community, w. (2025, April 4). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Five_why
- vi. Expo. (n.d.). *Expo-Router*. Retrieved from Expo documentation: <https://docs.expo.dev/router/introduction/>
- vii. F. N. Syamaidzar and J. Sutopo, J. I. (2023). Implementation of a location-based service on the geolocation presence system web-based, vol. 5, no.2, 2023. pp. pp. 45-55.
- viii. GeeksforGeeks. (n.d.). *System Design*. Retrieved from What is HLD: <https://www.geeksforgeeks.org/system-design/what-is-high-level-design-learn-system-design/>
- ix. Harsh Chavda, S. S. (March 2025). *Development of a Geo-Temporal Attendance Tracking Portal* .
- x. Hartono, T. H. (2024). The importance increasing attendance efficiency accuracy with presence system in era industrial revolution. *International Journal of Cyber and IT Service Management*, 133-142.
- xi. *Low Level Design*. (2025, May 2). Retrieved from [geeksforgeeks: https://www.geeksforgeeks.org/system-design/what-is-low-level-design-or-lll-learn-system-design/](https://www.geeksforgeeks.org/system-design/what-is-low-level-design-or-lll-learn-system-design/)

- xii. Muhamad Syamim, I. A. (n.d.). *The Development of a Mobile Application for a Reward-Based Student-Lecturer Appointment Using Ethereum Blockchain and Smart Contracts*, Faculty of Computer Science and Information Technology, U.
- xiii. Ohno, T. (1998). Toyota production system: beyond large-scale production. *Productivity Press*, 14.
- xiv. Othman, M. I. (2012). An adaptation of the web-based system architecture in the development of the online attendance system. . *IEEE Conference on Open Systems* (pp. pp. 1-6). IEEE.
- xv. Pankaj Tile, A. P. (2024). Geoworktrack: A comprehensive Real-time GPS-Based Employee Monitoring and Attendance Management System. *Scientific Research Journal of Science, Engineering and Technology ISSN: 2584-0584, Peer Reviewed Journal*, Vol-2 Issue-2 .
- xvi. Pathak, K. &. (2023.). *Attendance Monitoring System using Face Recognition*.
- xvii. *Requirement Elicitation*. (2025, April 20). Retrieved from geeksforgeeks: <https://www.geeksforgeeks.org/software-engineering/software-engineering-requirements-elicitation/>
- xviii. Ross Clarke, L. M. (2023). *Online voting scheme using ibm cloud-based hyperledger fabric with privacy-preservation*.
- xix. Serrat, O. D. (2025, April 4). Retrieved from ADB Publications: <https://www.adb.org/publications/five-whys-technique>

APPENDIX